[TÊN TRƯỜNG]
[TÊN KHOA]

LUẬN VĂN TỐT NGHIỆP

Design and Development of a Multi-Agent Platform for Object-Oriented Enterprise Workflow Automation

(Thiết kế và phát triển nền tảng đa tác tử tự động hoá quy trình doanh nghiệp theo hướng đối tượng)

Sinh viên thực hiện [Họ và tên] – [MSSV]
Giảng viên hướng dẫn [Học hàm, học vị, họ và tên]
Ngành [Tên ngành]

[Thành phố], tháng 12 năm 2026

LỜI CAM ĐOAN

[Sinh viên tự viết và ký: cam đoan luận văn là kết quả nghiên cứu của bản thân dưới sự hướng dẫn của giảng viên, các số liệu là trung thực và có nguồn gốc rõ ràng.]

LỜI CẢM ƠN

[Sinh viên tự viết.]

TÓM TẮT

Phát triển một dự án phần mềm cần nhiều vai trò phối hợp: phân tích nghiệp vụ, thiết kế kiến trúc, lập kế hoạch, lập trình, kiểm thử và kiểm soát chất lượng. Khi làm thủ công, tài liệu phân tích và thiết kế hướng đối tượng (OOAD) thường lệch khỏi mã nguồn, còn chất lượng được khẳng định thay vì được đo. Các hệ thống đa tác tử dùng mô hình ngôn ngữ lớn (LLM) gần đây tự động hoá được một phần chuỗi này, nhưng bản thân chúng hiếm khi được thiết kế theo OOAD và sản phẩm của chúng hiếm khi có truy vết từ yêu cầu đến kiểm thử.

Luận văn trình bày AI-CORP, một nền tảng đa tác tử gồm tám tác tử (CEO, BA, Architect, PM, DEV, QC, QA, Marketing) do một bộ máy quy trình (workflow engine) điều phối theo đồ thị có hướng không chu trình. Nền tảng được thiết kế theo kiến trúc phân lớp và thiết kế hướng miền (DDD): tầng miền độc lập framework, các tác tử là một lớp `BaseAgent` (Template Method) kết hợp hai chiến lược tư duy và hành động được tiêm vào (Strategy), vòng đời tác tử, công việc và quy trình là các máy trạng thái, các thành phần giao tiếp qua sự kiện miền (Observer). Điểm khác biệt là OOAD được áp dụng hai lần: cho chính nền tảng, và cho mọi dự án nền tảng sinh ra. Mỗi dự án đi qua chuỗi use case → mô hình miền → sơ đồ UML → mã phân lớp → kiểm thử chấp nhận theo use case → ma trận truy vết, với một cổng chất lượng OOAD sau thiết kế, phân tích tĩnh mã sinh ra bằng TypeScript compiler API và một vòng sửa lỗi có giới hạn trước khi QA duyệt.

Nền tảng còn có các tính năng doanh nghiệp (phân quyền theo vai trò và quyền sở hữu dự án, nhật ký kiểm toán chỉ ghi thêm, API tích hợp bằng khoá API, webhook ký HMAC, xuất báo cáo PDF/CSV/JSON), phân tích hiệu năng tác tử theo vai trò với cảnh báo bất thường, và mô hình hồi quy ridge dự báo thời gian và chi phí LLM của từng vai trò, được đánh giá bằng sai số leave-one-out.

Kết quả đánh giá: 674 kiểm thử tự động đạt trong 47 bộ; tầng miền đạt độ bao phủ 97,1% số dòng và 89,6% số nhánh; các lần chạy đầu-cuối trên PostgreSQL, Redis và nginx đạt 21/21, 21/21 và 44/44 kiểm tra; chi phí xử lý một nút quy trình (không tính độ trễ LLM) có trung vị 108 ms và phân vị 99 là 1,73 s; với 20 người dùng đồng thời API phục vụ 201,8 yêu cầu/giây, không lỗi. Kiểm thử đã phát hiện và sửa các lỗi thật, trong đó có một lỗ hổng cho phép người dùng chỉ có quyền xem duyệt quy trình qua WebSocket. Luận văn cũng nêu rõ giới hạn: các phép đo dùng một LLM kịch bản để

lặp lại được, nên chất lượng đầu ra với LLM thật và mức cải thiện nhờ cơ chế học của tác tử chưa được đo định lượng.

Từ khoá: phân tích và thiết kế hướng đối tượng, hệ đa tác tử, mô hình ngôn ngữ lớn, tự động hoá quy trình, mẫu thiết kế, truy vết yêu cầu.

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Nghĩa
ACL	Agent Communication Language
ADR	Architecture Decision Record (bản ghi quyết định kiến trúc)
API	Application Programming Interface
BA	Business Analyst (tác tử phân tích nghiệp vụ)
BDI	Belief–Desire–Intention
BPMN	Business Process Model and Notation
CQRS	Command Query Responsibility Segregation
DAG	Directed Acyclic Graph (đồ thị có hướng không chu trình)
DDD	Domain-Driven Design (thiết kế hướng miền)
DI	Dependency Injection
E2E	End-to-end (đầu-cuối)
FR / NFR	Functional / Non-functional Requirement
GoF	Gang of Four (Gamma, Helm, Johnson, Vlissides)
HMAC	Hash-based Message Authentication Code
JWT	JSON Web Token
LLM	Large Language Model (mô hình ngôn ngữ lớn)
LOO	Leave-one-out (kiểm định chéo bỏ một)
MAPE / MAE	Mean Absolute Percentage Error / Mean Absolute Error
MAS	Multi-Agent System (hệ đa tác tử)
OCP	Open/Closed Principle
OOAD	Object-Oriented Analysis and Design
QA / QC	Quality Assurance / Quality Control
RBAC	Role-Based Access Control
SOLID	SRP, OCP, LSP, ISP, DIP
SPA	Single-Page Application
UC	Use case
UML	Unified Modeling Language

MỤC LỤC

Lời cam đoan

Lời cảm ơn

Tóm tắt

Danh mục từ viết tắt

1 Mở đầu

1.1 Bối cảnh

1.2 Động lực và vấn đề nghiên cứu

1.3 Mục tiêu và phạm vi

1.4 Đóng góp

1.5 Phương pháp nghiên cứu

1.6 Cấu trúc luận văn

2 Cơ sở lý thuyết và tổng quan

2.1 Phân tích và thiết kế hướng đối tượng

2.2 Hệ đa tác tử

2.3 Điều phối quy trình

2.4 Trí tuệ nhân tạo trong kỹ nghệ phần mềm

2.5 Khoảng trống nghiên cứu

3 Phân tích yêu cầu

3.1 Tác nhân và vai trò tác tử

3.2 Yêu cầu chức năng

3.3 Yêu cầu phi chức năng

3.4 Mô hình use case

3.5 Đặc tả use case tiêu biểu

3.6 Mô hình miền

4 Thiết kế hệ thống

4.1 Kiến trúc tổng thể

4.2 Mẫu thiết kế

4.3 Thiết kế tác tử

4.4 Thiết kế bộ máy quy trình

4.5 Thiết kế dữ liệu và lưu trữ

4.6 Áp dụng OOAD cho dự án sinh ra

4.7 Bảo mật và tính năng doanh nghiệp
4.8 Phân tích, dự báo và chất lượng mã
4.9 Thiết kế triển khai

5 Hiện thực

5.1 Công nghệ và quy mô
5.2 Tầng miền
5.3 Tác tử
5.4 Bộ máy quy trình
5.5 Tính năng doanh nghiệp
5.6 Giao diện người dùng
5.7 Cấu hình triển khai

6 Kiểm thử và đánh giá

6.1 Chiến lược kiểm thử
6.2 Kiểm thử tự động và độ bao phủ
6.3 Kiểm thử đầu-cuối
6.4 Thời gian xử lý của một nút quy trình
6.5 Hiệu năng dưới tải
6.6 Độ chính xác của dự báo
6.7 Dự đoán chất lượng mã
6.8 Nghiên cứu tình huống
6.9 Lỗi phát hiện nhờ kiểm thử

7 Thảo luận

7.1 RQ1: OOAD có cải thiện kiến trúc của hệ đa tác tử dùng LLM không?
7.2 RQ2: Những mẫu thiết kế nào hiệu quả nhất cho việc điều phối?
7.3 RQ3: Tác tử học từ kết quả như thế nào, và đo điều đó ra sao?
7.4 Mối đe doạ đến tính hợp lệ của đánh giá
7.5 Các giới hạn của nền tảng

8 Kết luận và hướng phát triển

8.1 Kết luận
8.2 Hướng phát triển

Tài liệu tham khảo

Phụ lục A. Danh mục sơ đồ UML

Phụ lục B. Giao diện lập trình (API)

Phụ lục C. Số liệu và cách tái tạo kiểm thử

Kiểm thử tự động

Kiểm thử đầu-cuối

LLM kịch bản

Dữ liệu trình diễn

Phụ lục D. Số liệu hiệu năng

Kiểm thử tải

Thời gian theo bước tác tử

Phụ lục E. Hướng dẫn sử dụng

Cài đặt bằng Docker

Vai trò người dùng

Các thao tác chính

Chương 1
MỞ ĐẦU

1.1 Bối cảnh

Một dự án phần mềm doanh nghiệp, dù nhỏ, đi qua nhiều công đoạn do nhiều vai trò đảm nhận. Người phân tích nghiệp vụ thu thập yêu cầu và viết đặc tả use case; kiến trúc sư xây dựng mô hình miền, chọn mẫu thiết kế và vẽ các sơ đồ UML; người quản lý dự án chia việc; lập trình viên hiện thực; người kiểm thử viết và chạy kiểm thử; người đảm bảo chất lượng rà soát xem sản phẩm có đúng thiết kế và quy trình hay không. Phân tích và thiết kế hướng đối tượng (Object-Oriented Analysis and Design, OOAD) cung cấp ngôn ngữ chung cho chuỗi này: use case mô tả hành vi hệ thống từ góc nhìn tác nhân [3], mô hình miền và sơ đồ lớp mô tả cấu trúc, sơ đồ tuần tự và trạng thái mô tả hành vi [1], [2], còn các mẫu thiết kế ghi lại lời giải đã được kiểm chứng cho các vấn đề lặp lại [4], [5].

Trên thực tế, chuỗi này hay đứt gãy ở hai chỗ. Thứ nhất, tài liệu thiết kế lệch dần khỏi mã nguồn vì không có gì buộc chúng phải khớp nhau. Thứ hai, chất lượng thường được khẳng định thay vì được đo: một use case “đã xong” nhưng không có kiểm thử nào chứng minh, một lớp “theo kiến trúc phân lớp” nhưng tầng miền lại phụ thuộc hạ tầng.

Các mô hình ngôn ngữ lớn (LLM) đã cho thấy khả năng sinh mã [22] và giải một phần các tác vụ kỹ nghệ phần mềm thực tế [23]. Các hệ đa tác tử dựa trên LLM như MetaGPT [24], ChatDev [25] hay AutoGen [26] mô phỏng một nhóm phát triển với nhiều vai trò, mỗi vai trò là một tác tử. Tuy nhiên, các hệ thống này chủ yếu đo sản phẩm cuối (mã có chạy không) và ít quan tâm đến hai câu hỏi mà một luận văn về OOAD phải trả lời: bản thân hệ thống có được thiết kế tốt theo nguyên lý hướng đối tượng không, và sản phẩm nó tạo ra có truy vết được từ yêu cầu đến kiểm thử không.

1.2 Động lực và vấn đề nghiên cứu

Luận văn xuất phát từ một nền tảng đa tác tử ban đầu có năm tác tử (CEO, PM, DEV, QA, Marketing) và một bộ máy quy trình. Khi rà soát, nền tảng này có các vấn đề điển hình của một hệ thống phát triển nhanh: tầng miền được thiết kế nhưng không được dùng, lớp tác tử cơ sở gánh quá nhiều trách nhiệm, dịch vụ trộn logic nghiệp vụ với truy cập dữ liệu, không có kho lưu trữ (Repository) hay sự kiện miền, tài liệu UML

thiếu, và sản phẩm do tác tử tạo ra bị mất sau mỗi lần chạy vì được ghi vào một sandbox không lưu trữ.

Từ đó, luận văn đặt ra ba câu hỏi nghiên cứu:

- **RQ1.** Áp dụng có hệ thống các nguyên lý OOAD (phân lớp, SOLID, mẫu thiết kế, thiết kế hướng miền) có cải thiện được kiến trúc của một hệ đa tác tử dùng LLM về tính mô-đun, khả năng mở rộng và khả năng kiểm thử không?
- **RQ2.** Những mẫu thiết kế nào hiệu quả nhất cho việc điều phối các tác tử và quy trình của chúng?
- **RQ3.** Các tác tử có thể học từ những lần thực thi trước để cải thiện kết quả như thế nào, và làm sao đo được điều đó một cách trung thực?

1.3 Mục tiêu và phạm vi

Mục tiêu của luận văn là thiết kế và hiện thực một nền tảng đa tác tử mà:

1. Chính nó là một ví dụ mẫu về OOAD: có đặc tả use case, mô hình miền, đủ sáu góc nhìn UML, kiến trúc phân lớp, mẫu thiết kế được gắn với vấn đề cụ thể, và kiểm thử tự động đo được.
2. Mọi dự án nó tạo ra cũng tuân thủ OOAD và có truy vết từ use case đến kiểm thử.
3. Chất lượng được đo chứ không khẳng định: có cổng chất lượng, kiểm thử chấp nhận được thực thi, phân tích tĩnh, phân tích hiệu năng tác tử và dự báo được so với thực tế.
4. Có các tính năng cần cho môi trường doanh nghiệp: phân quyền, nhật ký kiểm toán, API tích hợp, xuất báo cáo và triển khai được.

Phạm vi được giới hạn có chủ đích:

- **Sản phẩm là dự án được phân tích, thiết kế, hiện thực và kiểm thử, không phải một bản triển khai đang chạy.** Vì vậy vai trò DevOps của nền tảng ban đầu bị loại bỏ (không có đầu ra kiểm chứng được và kéo phạm vi sang CI/CD), trong khi vai trò kiểm soát chất lượng (QC) được thêm vào và vai trò đảm bảo chất lượng (QA) được định nghĩa lại theo nghĩa quy trình (mục 3.1).
- **Dự án sinh ra viết bằng TypeScript** để nền tảng có thể tự phân tích tĩnh và chạy kiểm thử của chúng.

- **Các phép đo dùng một LLM kịch bản** (một máy chủ tương thích OpenAI trả lời theo kịch bản) để mọi kết quả lặp lại được. Nền tảng làm việc với bất kỳ cổng LLM tương thích OpenAI nào; đánh giá chất lượng đầu ra với LLM thật nằm ngoài phạm vi đo của luận văn và được nêu trong phần giới hạn.

1.4 Đóng góp

1. **Một kiến trúc tham chiếu cho hệ đa tác tử dùng LLM theo OOAD**: tầng miền độc lập framework (101 tệp), tác tử là Template Method cộng hai Strategy, máy trạng thái cho tác tử, công việc và quy trình, sự kiện miền cho giao tiếp giữa các mô-đun, Repository và cổng (port) cho mọi phụ thuộc hạ tầng.
2. **OOAD áp dụng cho sản phẩm của tác tử**: chuỗi use case → mô hình miền → UML → mã phân lớp → kiểm thử chấp nhận → ma trận truy vết, với cổng chất lượng OOAD, phân tích tĩnh có kiểm tra vi phạm phân lớp và một vòng tự sửa có giới hạn chi phí.
3. **Bộ máy quy trình có quy trình con (Composite), phiên bản bất biến, ảnh chụp và khôi phục (Memento), nhánh song song có phụ thuộc** và các mẫu quy trình có kiểu.
4. **Đánh giá trung thực**: phân tích hiệu năng tác tử theo tiêu chí riêng của từng vai trò, mô hình dự báo đánh giá bằng sai số leave-one-out, dự đoán phán quyết QA được so với phán quyết thật, cùng các lần chạy đầu-cuối có lưu vết.

1.5 Phương pháp nghiên cứu

Luận văn theo phương pháp nghiên cứu khoa học thiết kế (design science) [45]: xây dựng một tạo tác (nền tảng), đánh giá nó bằng các phép đo lặp lại được, và rút ra tri thức thiết kế. Việc phát triển đi theo vòng lặp: rà soát hiện trạng, phân tích yêu cầu thành use case, thiết kế, hiện thực theo hướng kiểm thử trước (TDD) [34], chạy kiểm thử đầu-cuối trên hạ tầng thật (PostgreSQL, Redis, nginx), và ghi lại mọi lỗi phát hiện được cùng cách sửa. Mỗi số liệu trong luận văn có nguồn trong kho mã (tệp kết quả hoặc lệnh để tái tạo).

1.6 Cấu trúc luận văn

Chương 2 trình bày cơ sở lý thuyết về OOAD, hệ đa tác tử, điều phối quy trình và ứng dụng AI trong kỹ nghệ phần mềm. Chương 3 phân tích yêu cầu và mô hình use case. Chương 4 trình bày thiết kế: kiến trúc, mẫu thiết kế, tác tử, bộ máy quy trình và cách nền tảng áp dụng OOAD cho dự án sinh ra. Chương 5 mô tả hiện thực. Chương 6 trình

bày kiểm thử và đánh giá. Chương 7 thảo luận câu trả lời cho các câu hỏi nghiên cứu và các giới hạn. Chương 8 kết luận và nêu hướng phát triển. Các phụ lục gồm danh mục sơ đồ UML, tài liệu API, số liệu kiểm thử, số liệu hiệu năng và hướng dẫn sử dụng.

Chương 2
CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN

Chương này trình bày bốn mảng kiến thức mà nền tảng dựa vào: phân tích và thiết kế hướng đối tượng, hệ đa tác tử, điều phối quy trình, và ứng dụng trí tuệ nhân tạo trong kỹ nghệ phần mềm. Cuối chương nêu khoảng trống mà luận văn hướng tới.

2.1 Phân tích và thiết kế hướng đối tượng

2.1.1 Mô hình hoá bằng use case và UML

Jacobson đề xuất phát triển phần mềm hướng use case [3]: mỗi use case là một chuỗi tương tác giữa một tác nhân (actor) và hệ thống mang lại một kết quả có giá trị cho tác nhân đó. Use case vừa là đơn vị yêu cầu, vừa là đơn vị kiểm thử chấp nhận và là trục để truy vết đến thiết kế và mã. Các quan hệ `«include»` (hành vi con bắt buộc), `«extend»` (hành vi mở rộng có điều kiện) và tổng quát hoá giúp tránh lặp lại đặc tả.

Ngôn ngữ mô hình hoá thống nhất UML [1], [2] hợp nhất ký pháp của Booch, Rumbaugh và Jacobson. UML 2.5 chia sơ đồ thành hai nhóm: **cấu trúc** (lớp, đối tượng, thành phần, triển khai, gói, cấu trúc hỗn hợp) và **hành vi** (use case, tuần tự, hoạt động, máy trạng thái, giao tiếp, thời gian, tổng quan tương tác). Kruchten tổ chức tài liệu kiến trúc theo mô hình 4+1 góc nhìn (logic, tiến trình, phát triển, vật lý, cộng thêm kịch bản) [12]; luận văn dùng cách tổ chức tương tự với sáu nhóm sơ đồ: use case, lớp, tuần tự, hoạt động, máy trạng thái và kiến trúc (gói, thành phần, triển khai).

Larman mô tả một quy trình OOAD lặp: use case → mô hình miền (khái niệm, thuộc tính, liên kết, chưa có phương thức) → sơ đồ tuần tự hệ thống → hợp đồng thao tác → thiết kế lớp với các nguyên tắc gán trách nhiệm GRASP (Information Expert, Creator, Controller, Low Coupling, High Cohesion…) [9]. Nền tảng áp dụng đúng chuỗi này cho dự án sinh ra: tác tử BA viết use case, tác tử Architect dựng mô hình miền, sơ đồ lớp, sơ đồ tuần tự và chọn mẫu thiết kế.

2.1.2 Các nguyên lý SOLID và mô-đun hoá

Parnas chỉ ra rằng tiêu chí phân rã mô-đun đúng là che giấu những quyết định dễ thay đổi [55]. Martin hệ thống hoá năm nguyên lý thiết kế lớp, gọi tắt là SOLID [7], [8]:

- **SRP** (Single Responsibility): một lớp chỉ có một lý do để thay đổi.
- **OCP** (Open/Closed), bắt nguồn từ Meyer [10]: mở để mở rộng, đóng để sửa đổi.

- **LSP** (Liskov Substitution) [11]: lớp con thay thế được lớp cha mà không phá vỡ hành vi mong đợi.
- **ISP** (Interface Segregation): nhiều giao diện nhỏ chuyên biệt tốt hơn một giao diện lớn.
- **DIP** (Dependency Inversion): mô-đun cấp cao và cấp thấp đều phụ thuộc vào trừu tượng.

Kiến trúc sạch (Clean Architecture) của Martin [8] và kiến trúc phân lớp của Evans [6] cùng đưa ra **quy tắc phụ thuộc**: mã ở vòng trong (miền) không biết gì về vòng ngoài (ứng dụng, hạ tầng, giao diện). Nền tảng dùng quy tắc này ở hai nơi: cho chính mã của nó, và như một luật được kiểm tra tự động trên mã do tác tử DEV sinh ra (mục 4.6).

2.1.3 Mẫu thiết kế

Gamma và cộng sự (GoF) phân loại 23 mẫu thành ba nhóm: khởi tạo (Factory Method, Abstract Factory, Builder, Prototype, Singleton), cấu trúc (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) và hành vi (Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor) [4]. Fowler bổ sung các mẫu cho ứng dụng doanh nghiệp như Repository, Data Mapper, Service Layer, Unit of Work, Domain Model [5]. Evans thêm các khối xây dựng của thiết kế hướng miền: Entity, Value Object, Aggregate, Domain Event, Repository, Factory, Domain Service [6].

Một mẫu chỉ có giá trị khi gắn với một vấn đề cụ thể trong ngữ cảnh. Vì vậy luận văn trình bày mẫu thiết kế dưới dạng bảng "vấn đề → mẫu → lớp hiện thực" (mục 4.2) thay vì liệt kê số lượng.

2.1.4 Truy vết yêu cầu

Gotel và Finkelstein định nghĩa truy vết yêu cầu là khả năng theo dõi vòng đời của một yêu cầu theo cả hai chiều, từ nguồn gốc đến đặc tả, thiết kế, hiện thực và kiểm thử [44]. IEEE 830 coi tính truy vết được là một thuộc tính của đặc tả yêu cầu tốt [42]. Ma trận truy vết là công cụ phổ biến nhất; giá trị của nó phụ thuộc vào việc các ô được cập nhật từ dữ liệu thật (kiểm thử đã chạy) hay chỉ được điền tay. Nền tảng xây ma trận từ kết quả thực thi kiểm thử chấp nhận (mục 4.6).

2.1.5 Chất lượng phần mềm: QA và QC

ISO 9000 phân biệt **đảm bảo chất lượng** (QA, tạo niềm tin rằng yêu cầu chất lượng sẽ được đáp ứng, tức hướng quy trình và phòng ngừa) với **kiểm soát chất lượng** (QC,

đáp ứng yêu cầu chất lượng, tức hướng sản phẩm và phát hiện) [53]. ISO/IEC 25010 định nghĩa mô hình chất lượng sản phẩm với các đặc tính như tính phù hợp chức năng, hiệu năng, bảo mật, khả năng bảo trì (gồm tính mô-đun, khả năng tái sử dụng, khả năng phân tích, sửa đổi và kiểm thử) [43]. Sự phân biệt QA/QC là cơ sở để nền tảng tách hai tác tử với trách nhiệm khác nhau (mục 3.1).

2.2 Hệ đa tác tử

2.2.1 Tác tử và kiến trúc tác tử

Wooldridge định nghĩa tác tử là một hệ thống máy tính đặt trong một môi trường, có khả năng hành động tự chủ để đạt mục tiêu; tác tử thông minh có tính phản ứng, tính chủ động và năng lực xã hội [14]. Kiến trúc BDI (Belief–Desire–Intention) của Rao và Georgeff mô hình hoá tác tử bằng niềm tin về thế giới, mong muốn và ý định đã cam kết [15]. Chu trình "nhận thức – suy nghĩ – hành động" của BDI tương ứng trực tiếp với cặp phương thức `think()` / `act()` của tác tử trong nền tảng, trong đó bộ nhớ dài hạn đóng vai trò niềm tin và mục tiêu của công việc đóng vai trò mong muốn.

2.2.2 Giao tiếp và phối hợp

FIPA chuẩn hoá ngôn ngữ giao tiếp tác tử (ACL) với các hành động ngôn từ (performative) như `inform`, `request`, `propose`, `accept-proposal`, `reject-proposal` [16]. Giao thức Contract Net của Smith là mô hình kinh điển cho phân công việc: người quản lý gọi thầu, các tác tử đề xuất, người quản lý trao hợp đồng [17]. Stone và Veloso phân loại hệ đa tác tử theo mức độ đồng nhất và mức độ giao tiếp giữa các tác tử [18].

Nền tảng chọn cách phối hợp **có điều phối trung tâm**: bộ máy quy trình quyết định tác tử nào làm gì theo một đồ thị đã định nghĩa, còn các tác tử trao đổi thông điệp có kiểu (INFORM, REQUEST, PROPOSE, ACCEPT, REJECT) qua một bus thông điệp (Mediator) trong các phiên cộng tác như lập trình cặp. Lựa chọn này đánh đổi tính tự chủ lấy tính dự đoán được và khả năng truy vết, phù hợp với mục tiêu của một quy trình phần mềm có cổng chất lượng.

2.3 Điều phối quy trình

BPMN 2.0 là chuẩn ký pháp cho quy trình nghiệp vụ, với sự kiện, công việc, cổng (gateway) rẽ nhánh và song song [19]. Mạng Petri cung cấp nền tảng hình thức để phân tích tính đạt được, tính sống và tính bị chặn của quy trình [20]. Van der Aalst và

cộng sự tổng hợp các mẫu quy trình (workflow patterns) như tuần tự, tách song song, đồng bộ, lựa chọn độc quyền, vòng lặp tuỳ ý và quy trình con [21]. Statecharts của Harel mở rộng máy trạng thái hữu hạn với trạng thái lồng và song song [33], là cơ sở cho sơ đồ máy trạng thái của UML.

Bộ máy quy trình của nền tảng hiện thực tập con các mẫu cần cho phát triển phần mềm: tuần tự, lựa chọn có điều kiện (theo phán quyết của QA), vòng lặp (QA từ chối → DEV), tách và đồng bộ song song có phụ thuộc, quy trình con, và điểm dừng chờ con người duyệt. Đồ thị được kiểm tra không chu trình bằng sắp xếp tô-pô, trừ các cạnh quay lui được khai báo tường minh.

2.4 Trí tuệ nhân tạo trong kỹ nghệ phần mềm

2.4.1 Mô hình ngôn ngữ lớn và sinh mã

Kiến trúc Transformer [32] là nền tảng của các LLM hiện nay. Chen và cộng sự giới thiệu Codex và bộ đánh giá HumanEval cho bài toán sinh hàm từ mô tả [22]. SWE-bench đo khả năng giải các issue thật trên GitHub và cho thấy khoảng cách lớn giữa sinh một hàm và sửa một kho mã [23]. Gợi ý chuỗi suy luận (chain-of-thought) cải thiện khả năng suy luận nhiều bước [30]; ReAct xen kẽ suy luận và hành động gọi công cụ [27]; sinh tăng cường truy xuất (RAG) đưa tri thức bên ngoài vào ngữ cảnh [31].

2.4.2 Tác tử LLM và hệ đa tác tử cho phát triển phần mềm

MetaGPT mã hoá quy trình vận hành chuẩn (SOP) của một công ty phần mềm thành các vai trò (quản lý sản phẩm, kiến trúc sư, kỹ sư…) trao đổi tài liệu có cấu trúc [24]. ChatDev tổ chức phát triển thành các giai đoạn thiết kế, lập trình, kiểm thử, mỗi giai đoạn là cuộc hội thoại giữa hai vai trò [25]. AutoGen cung cấp khung lập trình cho hội thoại nhiều tác tử [26]. Về học từ kinh nghiệm, Reflexion cho tác tử tự viết nhận xét về thất bại và đưa vào lần thử sau [28]; Generative Agents lưu ký ức, truy xuất theo độ liên quan và tổng hợp thành suy ngẫm [29].

Cơ chế học của nền tảng gần với Reflexion và Generative Agents: mỗi kết quả của tác tử được chấm bởi một tín hiệu khách quan (cổng OOAD, kết quả kiểm thử của QC, phán quyết của QA), và định kỳ các kết quả được tinh luyện thành bài học lưu trong bộ nhớ của vai trò đó để đưa vào lời nhắc lần sau (mục 4.3.4).

2.4.3 Đo chất lượng mã và ước lượng công sức

McCabe định nghĩa độ phức tạp chu trình (cyclomatic complexity) bằng số đường độc lập qua đồ thị luồng điều khiển [35]. Halstead đo thể tích chương trình từ số toán tử và toán hạng [36]. Oman, Hagemeister và Coleman kết hợp hai độ đo này với số dòng thành chỉ số bảo trì (Maintainability Index) [37], [38]. Về ước lượng, COCOMO của Boehm là mô hình hồi quy kinh điển giữa kích thước và công sức [39]; hồi quy ridge [40] thêm phạt chuẩn L2 để ổn định khi dữ liệu ít và các biến tương quan; kiểm định chéo bỏ một (leave-one-out) cho ước lượng ít thiên lệch về sai số tổng quát khi mẫu nhỏ [46].

2.5 Khoảng trống nghiên cứu

Các hệ đa tác tử cho phát triển phần mềm hiện có [24]–[26] tập trung vào việc sinh ra mã chạy được. Ba khía cạnh ít được đề cập:

1. **Bản thân hệ thống** hiếm khi được trình bày như một sản phẩm OOAD có mô hình miền, mẫu thiết kế gắn với vấn đề, và kiểm thử đo được độ bao phủ.
2. **Sản phẩm của tác tử** thường thiếu chuỗi truy vết từ use case qua thiết kế đến kiểm thử đã thực thi, và thiếu cổng kiểm tra rằng mã tuân thủ kiến trúc đã thiết kế.
3. **Đánh giá** thường dựa trên tỉ lệ thành công tổng thể, ít khi đo riêng từng vai trò theo tiêu chí phù hợp với vai trò đó, và ít khi so dự báo với kết quả thực tế.

Luận văn hướng tới cả ba khía cạnh này, với giới hạn được nêu ở mục 1.3: các phép đo dùng LLM kịch bản để lặp lại được.

Chương 3
PHÂN TÍCH YÊU CẦU

Chương này xác định ai dùng nền tảng, nền tảng phải làm gì và phải đạt chất lượng nào. Đặc tả đầy đủ 31 use case nằm trong `docs/requirements/use-cases.md`; ma trận truy vết yêu cầu → use case → thiết kế → mã → kiểm thử nằm trong `docs/requirements/traceability-matrix.md`. Chương này tóm tắt và phân tích các quyết định chính.

3.1 Tác nhân và vai trò tác tử

Nền tảng có hai loại tác nhân: con người và tác tử AI.

Tác nhân con người. Phân quyền theo vai trò (RBAC) [47] với ba vai trò:

Vai trò	Quyền chính
Quản trị viên (admin)	Quản lý người dùng và vai trò, cấu hình mô hình LLM cho tác tử, xem nhật ký kiểm toán, quản lý khoá API và webhook của mọi người, theo dõi sức khoẻ hệ thống
Thành viên hội đồng quản trị (board member)	Tạo và sở hữu dự án, duyệt hoặc từ chối các bước cần con người, xem chi phí, báo cáo và phân tích, xuất dữ liệu, quản lý khoá API và webhook của mình
Người xem (viewer)	Chỉ đọc: xem dự án, báo cáo và tài liệu sinh ra

Ngoài vai trò, mỗi dự án có một **chủ sở hữu**: thành viên hội đồng chỉ duyệt được các phê duyệt được giao cho mình và chỉ sửa được dự án của mình. Tác tử AI không phải người dùng đăng nhập; mọi hành động của chúng được ghi vào nhật ký kiểm toán với loại tác nhân `agent`.

Tác tử AI. Nền tảng ban đầu có năm tác tử. Việc rà soát vai trò dẫn đến ba thay đổi:

1. **Thêm BA và Architect**, vì một quy trình OOAD cần người viết use case và người dựng mô hình miền trước khi lập kế hoạch.
2. **Bỏ DevOps.** Sản phẩm của nền tảng là một dự án đã được phân tích, thiết kế, hiện thực và kiểm thử, không phải một bản triển khai đang chạy. Tác tử DevOps chưa từng nằm trong quy trình, chỉ sinh văn bản không ai dùng, và sandbox không có mạng hay ổ lưu trữ để triển khai. Giữ lại nó sẽ thêm một vai trò không có đầu ra kiểm chứng được và kéo phạm vi sang CI/CD.

3. **Tách QA và QC** theo định nghĩa của ISO 9000 [53]. Tác tử QA cũ trộn cả hai việc và thực tế không làm tốt việc nào.

Tác tử	Trách nhiệm	Đầu ra kiểm chứng được
CEO	Định hướng, chia mốc, đóng dự án	Mốc (milestone), báo cáo đóng dự án
BA	Phân tích yêu cầu	`docs/use-cases.md` (use case có ID, tác nhân, luồng chính, luồng thay thế, tiêu chí chấp nhận)
Architect	Thiết kế	Mô hình miền, sơ đồ use case, lớp, tuần tự (PlantUML), `docs/architecture-patterns.md`, khung mã phân lớp
PM	Lập kế hoạch	Một công việc cho mỗi use case đã duyệt, có ước lượng chi phí
DEV	Hiện thực	Mã TypeScript theo bốn tầng, chú thích `UC-xxx`
QC (sản phẩm)	Kiểm soát chất lượng	`tests/UC-xxx.spec.ts`, kết quả chạy Jest, `docs/traceability-matrix.md`, độ bao phủ use case
QA (quy trình)	Đảm bảo chất lượng	Cổng OOAD sau thiết kế, rà soát mã so với thiết kế, phán quyết PASS/FAIL điều khiển vòng lặp về DEV
Marketing	Truyền thông	Thông báo sản phẩm

3.2 Yêu cầu chức năng

Các yêu cầu chức năng được nhóm thành bốn nhóm lớn (mã trong ngoặc là nhóm trong ma trận truy vết):

Hệ thống tác tử (FR1). Tám vai trò tác tử như trên; tác tử học từ kết quả trước qua bộ nhớ được tinh luyện; tác tử cộng tác theo các mẫu lập trình cặp và chuỗi rà soát; tác tử giao việc con cho nhau qua bus thông điệp.

Bộ máy quy trình (FR2). Quy trình con và lồng ghép quy trình; phiên bản bất biến và khôi phục về ảnh chụp; thực thi song song có giải quyết phụ thuộc; mẫu quy trình có tham số kiểu.

Tính năng doanh nghiệp (FR3). RBAC với quyền chi tiết và quyền sở hữu dự án; nhật ký kiểm toán đầy đủ; API tích hợp cho công cụ bên ngoài (khoá API, webhook); xuất dữ liệu và báo cáo.

Tính năng AI/ML (FR4). Phân tích hiệu năng và quá trình học của tác tử; mô hình ước lượng dựa trên dữ liệu lịch sử; dự đoán chất lượng mã bằng phân tích tĩnh; gợi ý tối ưu dựa trên bất thường.

Ma trận truy vết có 120 yêu cầu chức năng và phi chức năng: 111 ở trạng thái *Implemented*, 1 ở *Partial* và 8 ở *Planned* (các mục chưa làm được ghi rõ là chưa làm thay vì bị xoá); mỗi yêu cầu đã hiện thực trỏ đến tệp mã và tệp kiểm thử.

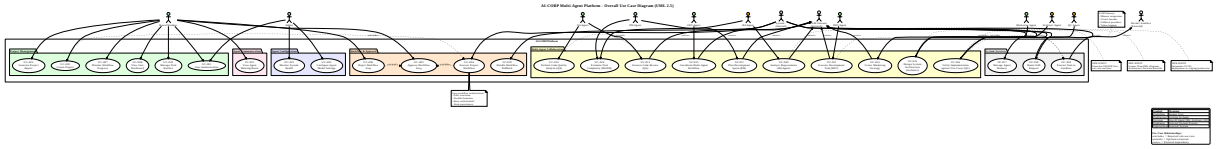
3.3 Yêu cầu phi chức năng

Nhóm	Yêu cầu	Cách kiểm chứng
Kiến trúc	Tuân thủ SOLID; mẫu thiết kế được áp dụng có lý do; tách tầng miền, ứng dụng, hạ tầng; tiêm phụ thuộc xuyên suốt	Rà soát thiết kế (chương 4), luật phân lớp được kiểm tra tự động trên mã sinh ra
Kiểm thử	Tầng miền ≥ 90% hàm, dòng, lệnh và ≥ 85% nhánh; toàn API có ngưỡng sàn; kiểm thử đầu-cuối cho các hành trình chính	Ngưỡng `coverageThreshold` của Jest làm hỏng lệnh `test:cov` nếu không đạt; các lần chạy đầu-cuối có lưu vết
Hiệu năng	Mỗi nút quy trình ≤ 2 s (không tính độ trễ LLM); các đọc trên bảng điều khiển trong ngân sách p50/p95/p99 với 20 người dùng đồng thời, lỗi ≤ 1%	Đo bước quy trình (mục 6.4), kiểm thử tải có ngân sách (mục 6.5)
Bảo mật	Băm mật khẩu bcrypt; JWT có hạn; khoá API chỉ mang phạm vi mà vai trò chủ sở hữu cho phép; nhật ký kiểm toán không lưu bí mật; webhook ký HMAC và chặn địa chỉ mạng nội bộ; sandbox cách ly; cấu hình production sai thì không khởi động	Kiểm thử đơn vị và đầu-cuối (mục 6.3)
Tin cậy	Thử lại có giãn cách mũ; khôi phục quy trình khi khởi động lại; kiểm tra sức khoẻ cho cân bằng tải	Kiểm thử bộ máy quy trình, kiểm thử sức khoẻ
Tài liệu	Đủ sơ đồ use case, lớp, tuần tự, hoạt động, máy trạng thái, thành phần và triển khai	39 sơ đồ PlantUML được render tự động, lệnh render thất bại nếu có lỗi cú pháp

3.4 Mô hình use case

Nền tảng có 31 use case được đặc tả, chia thành bốn nhóm.

Nhóm	Use case
Quản lý dự án (con người)	UC-001 Xác thực · UC-002 Tạo dự án · UC-005 Duyệt bước quy trình · UC-006 Từ chối bước quy trình · UC-007 Theo dõi tiến độ · UC-008 Xem chi phí · UC-020 Quản lý Kanban · UC-022 Tạo báo cáo dự án · UC-026 Duyệt sản phẩm của dự án
Tác tử	UC-009 Phân tích yêu cầu (BA) · UC-010 Thiết kế kiến trúc (Architect) · UC-011 Lập kế hoạch (PM) · UC-012 Thực hiện công việc phát triển (DEV) · UC-013 Rà soát mã (QA) · UC-014 Chiến lược marketing · UC-015 Điều phối quy trình (CEO) · UC-016 Kiểm chứng hiện thực theo use case (QC) · UC-023 Phân tích chất lượng mã (DEV, QA)
Hệ thống	UC-004 Thực thi quy trình · UC-017 Quản lý bộ nhớ tác tử · UC-018 Định tuyến yêu cầu LLM · UC-019 Khôi phục quy trình · UC-021 Phòng họp tác tử · UC-024 Dự báo công sức và chi phí
Quản trị và tích hợp	UC-003 Cấu hình tác tử · UC-025 Theo dõi sức khoẻ hệ thống · UC-029 Quản lý người dùng và vai trò · UC-030 Xem nhật ký kiểm toán · UC-031 Quản lý khoá API và webhook · UC-032 Gọi API tích hợp · UC-033 Xem phân tích hiệu năng tác tử



Hình 3.1. *Sơ đồ use case tổng thể của nền tảng*

Các quan hệ giữa use case phản ánh cấu trúc quy trình: UC-004 *Thực thi quy trình* «include» UC-005 *Duyệt bước* khi quy trình có nút chờ con người; UC-006 *Từ chối* «extend» UC-005; UC-012 *Phát triển* «include» UC-023 *Phân tích chất lượng mã* (dự đoán phán quyết QA và sửa một vòng); UC-016 của QC dùng kết quả của UC-009 (use case do BA viết) làm đầu vào.

3.5 Đặc tả use case tiêu biểu

3.5.1 UC-002 Tạo dự án

- **Tác nhân:** thành viên hội đồng quản trị.

- **Tiền điều kiện:** đã đăng nhập, có quyền `project:create`.
- **Luồng chính:** (1) người dùng chọn mẫu quy trình và điền tham số (ví dụ số use case tối thiểu, có bật bước duyệt của con người không); (2) hệ thống kiểm tra mẫu và tham số trước khi tạo bất cứ thứ gì; (3) thực thể `Project` được tạo qua phương thức nhà máy `Project.create`, áp dụng các bất biến (tên không rỗng, ngân sách dương); (4) dự án được lưu, sự kiện `ProjectCreatedEvent` được phát; (5) bộ máy quy trình tạo một lượt chạy gắn với phiên bản bất biến `mẫu@phiên_bản+băm` của định nghĩa quy trình; (6) một dự báo thời gian và chi phí được lưu khi quy trình bắt đầu.
- **Luồng thay thế:** mẫu không tồn tại → 404; tham số sai kiểu → 400 trước khi tạo dự án; người xem gọi → 403 và một mục `access.denied` trong nhật ký kiểm toán.
- **Hậu điều kiện:** dự án ở trạng thái `active`, có một lượt chạy quy trình; nhật ký kiểm toán có `project.create` và `workflow.started`.

3.5.2 UC-004 Thực thi quy trình

- **Tác nhân:** hệ thống (bộ máy quy trình) và các tác tử.
- **Luồng chính:** bộ máy lấy nút hiện tại, chọn chiến lược thực thi theo loại nút (`agent_task`, `sub_workflow`, `quality_gate`, `condition`, `parallel`, `human_approval`), ghi một bước (`WorkflowStep`) với thời lượng, gộp kết quả vào ngữ cảnh lượt chạy, chụp ảnh trạng thái (Memento), chọn cạnh kế tiếp và lặp lại đến nút kết thúc.
- **Luồng thay thế:** nút thất bại được thử lại tối đa ba lần với giãn cách mũ; nếu vẫn thất bại, lượt chạy đi theo cạnh `onFail` hoặc chuyển sang `failed` và phát `workflow.failed`; nút chờ duyệt chuyển lượt chạy sang `paused`; ngân sách vượt ngưỡng tạm dừng lượt chạy.

3.5.3 UC-016 Kiểm chứng hiện thực theo use case (QC)

- **Tác nhân:** tác tử QC.
- **Luồng chính:** (1) QC đọc các use case BA đã viết và mã DEV đã viết; (2) viết một tệp kiểm thử Jest cho mỗi use case (`tests/UC-xxx.spec.ts`); (3) hệ thống chạy kiểm thử trong sandbox Docker, hoặc trong một bản sao tạm khi được phép chạy cục bộ; (4) QC dựng ma trận truy vết use case → lớp → tệp hiện thực → kiểm thử và kết quả; (5) độ bao phủ use case được ghi vào siêu dữ liệu OOAD của dự án.

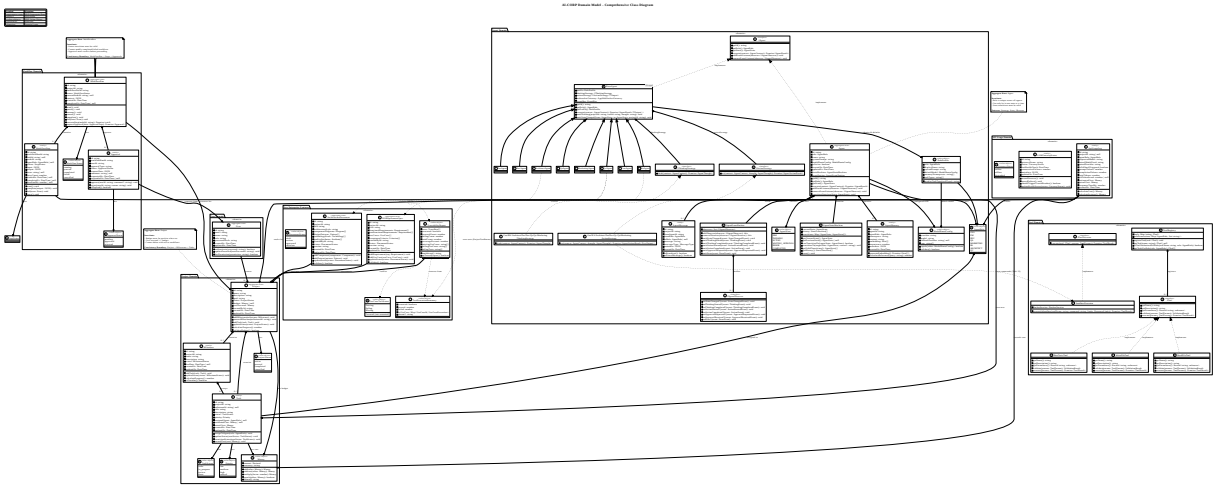
- **Hậu điều kiện:** độ bao phủ phản ánh **kiểm thử đã chạy đạt**, không phải kiểm thử đã được viết. Use case không có kiểm thử được liệt kê là thiếu.

3.6 Mô hình miền

Mô hình miền của nền tảng gồm ba aggregate chính và các khái niệm hỗ trợ:

- **Project** (gốc aggregate) chứa các `Milestone` và `Task`; có ngân sách và chi phí tích luỹ dạng giá trị `Money`; trạng thái `ProjectStatus` là một value object có máy trạng thái.
- **Agent** có `AgentRole`, cấu hình mô hình, bộ nhớ (`AgentMemory`) và máy trạng thái vòng đời.
- **WorkflowRun** tham chiếu một phiên bản định nghĩa quy trình bất biến, chứa các `WorkflowStep`, các ảnh chụp `WorkflowSnapshot`, có thể có lượt chạy cha (quy trình con) và các `Approval`.

Các khái niệm hỗ trợ: `User` và `Role`, `AuditLog` (chỉ ghi thêm), `ApiKey`, `Webhook` và `WebhookDelivery`, `ApiUsageLog` (chi phí LLM), `QualityGateLog`, `ProjectEstimate`.



Hình 3.2. *Mô hình miền tổng quan*

Chương 4
THIẾT KẾ HỆ THỐNG

Chương này trình bày thiết kế của nền tảng theo thứ tự từ tổng thể đến chi tiết: kiến trúc, mẫu thiết kế, tác tử, bộ máy quy trình, dữ liệu, cách áp dụng OOAD cho dự án sinh ra, bảo mật, phân tích và dự báo, và triển khai. Toàn bộ 39 sơ đồ UML được liệt kê ở Phụ lục A.

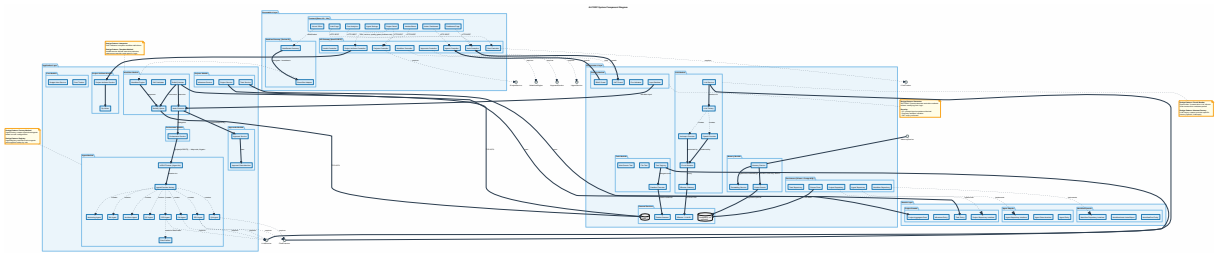
4.1 Kiến trúc tổng thể

4.1.1 Kiểu kiến trúc

Nền tảng gồm một máy chủ API (NestJS, TypeScript) và một ứng dụng web một trang (React). Bên trong API, mã được tổ chức theo **kiến trúc phân lớp kết hợp thiết kế hướng miền** [6], [8]:

Tầng	**Nội dung**	**Phụ thuộc được phép**
Trình bày	Controller REST, cổng WebSocket (Socket.IO), DTO, guard	Ứng dụng
Ứng dụng	Dịch vụ ứng dụng (một use case một phương thức), chiến lược tác tử, bộ máy quy trình, trình xử lý sự kiện	Miền
Miền (`src/domain`)	Thực thể, value object, aggregate, sự kiện miền, máy trạng thái, giao diện Repository và các cổng (port), chính sách thuần (không I/O)	Không gì cả
Hạ tầng	Repository Prisma, nhà cung cấp LLM, sandbox Docker, kho tệp của dự án, phân tích mã TypeScript, xuất PDF	Miền (hiện thực các cổng)

Tầng miền có 101 tệp và không nhập NestJS, Prisma hay bất kỳ thư viện I/O nào. Nhờ đó mọi quy tắc nghiệp vụ (chuyển trạng thái công việc, phép tính tiền, phát hiện bất thường, hồi quy dự báo, chính sách chất lượng mã) được kiểm thử như hàm thuần, và đây cũng là nơi đặt ngưỡng độ bao phủ cao nhất (mục 6.2).



Hình 4.1. *Sơ đồ thành phần*

4.1.2 Các quyết định kiến trúc chính

Các quyết định được ghi thành bản ghi quyết định kiến trúc (ADR) trong `docs/design/architecture.md`. Sáu quyết định có ảnh hưởng lớn nhất:

1. **NestJS cho API** vì hệ thống tiêm phụ thuộc và mô-đun của nó hiện thực trực tiếp DIP và cho phép đăng ký nhiều hiện thực cho một token (dùng cho danh sách tác tử và chiến lược nút).
2. **PostgreSQL** cho dữ liệu quan hệ và JSON (ngữ cảnh lượt chạy, siêu dữ liệu OOAD), với **Prisma** làm lớp ánh xạ phía sau các Repository.
3. **BullMQ trên Redis** cho hàng đợi nút quy trình, với chế độ dự phòng thực thi trực tiếp khi không có Redis (sức khoẻ hệ thống báo *degraded* thay vì ngừng).
4. **Repository** cho mọi truy cập dữ liệu từ tầng ứng dụng, để dịch vụ ứng dụng kiểm thử được bằng Repository trong bộ nhớ.
5. **Strategy cho chuyên môn hoá tác tử** thay vì kế thừa sâu (mục 4.3).
6. **Sự kiện miền** cho giao tiếp giữa các mô-đun: nhật ký kiểm toán, webhook, học của tác tử, dự báo và giám sát bất thường đều là người nghe sự kiện, không mô-đun nào gọi trực tiếp mô-đun kia.

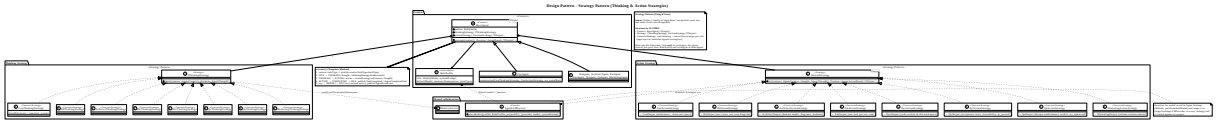
4.2 Mẫu thiết kế

Bảng sau gắn mỗi mẫu với vấn đề nó giải quyết và lớp hiện thực. Mỗi lớp trong mã có chú thích `@pattern` tương ứng, nên bảng này có thể kiểm chứng bằng tìm kiếm trong kho mã.

Vấn đề	Mẫu	Hiện thực
Tám vai trò cùng một chu trình nhận việc → suy nghĩ → hành động → hoàn tất	Template Method	`BaseAgent.execute()`
Mỗi vai trò suy nghĩ và hành động khác nhau, cần thay được độc lập	Strategy	`IThinkingStrategy`, `IActionStrategy`, 16 lớp chiến lược

Vấn đề	Mẫu	Hiện thực
Thêm vai trò mới không sửa bộ điều phối	Registry + DIP	`AgentOrchestrator` nhận danh sách `IAgent` qua token `AGENTS`; `RoleProfile` cho mỗi vai trò
Vòng đời tác tử, công việc, dự án, phê duyệt có chuyển trạng thái hợp lệ	State	`AgentStateMachine`, `Task.transitionTo`, value object `ProjectStatus`
Nhiều bên cần biết khi tác tử đổi trạng thái hoặc hoàn tất	Observer	`AgentEventEmitter` / `IAgentObserver`; `EventBus` với các handler
Mỗi loại nút quy trình thực thi khác nhau, thêm loại nút không sửa bộ máy	Strategy (tự đăng ký)	`NodeExecutionStrategy`: tác tử, quy trình con, cổng chất lượng, điều kiện, song song, phê duyệt
Một nút là cả một quy trình	Composite	`SubWorkflowStrategy` chạy quy trình con như lượt chạy con
Quay lại trạng thái trước của lượt chạy	Memento	`WorkflowRunMemento`, `WorkflowSnapshotCaretaker`
Định nghĩa quy trình không bị sửa sau khi đã có lượt chạy dùng nó	Immutable registry	`WorkflowDefinitionRegistry` theo `id@phiên_bản+băm`
Ghi chi phí mọi lời gọi LLM mà không sửa nhà cung cấp	Decorator	`UsageTrackingLLMProvider`
Tác tử gọi LLM qua một điểm duy nhất (cấu hình, nhà cung cấp, chi phí)	Facade	`AgentLLMService`, `OoadArtifactService`, `CodeQualityService`
Nhiều nhà cung cấp LLM sau một giao diện	Adapter + Factory Method	`ILLMProvider`, `LLMProviderFactory`
Tác tử trao đổi thông điệp có kiểu khi cộng tác	Mediator	`AgentMessageBus`, `EventBus`
Xác thực → giới hạn tần suất → kiểm quyền	Chain of Responsibility	`AuthenticationGuard` → `RateLimitGuard` → `PermissionGuard`; chuỗi rà soát QC → QA
Sự kiện miền thành mục kiểm toán hoặc payload webhook công khai	Adapter	`AuditLogHandler`, `webhook-payloads`
Truy cập dữ liệu tách khỏi miền	Repository, Data Mapper	`I*Repository` trong miền, `Prisma*Repository` trong hạ tầng
Báo cáo chi phí và hiệu năng dựng từng bước	Builder	`CostReportBuilder`, `PerformanceReportBuilder`
Xuất JSON / CSV / PDF chọn theo định dạng	Strategy + Registry	`ProjectExportService` và ba exporter

Vấn đề	Mẫu	Hiện thực
Mỗi loại bất thường, mỗi tín hiệu chất lượng là một quy tắc	Strategy + Composite	`IAgentAnomalyRule`, `IQualitySignal`, `AgentAnomalyDetector`
Phân tích mã sinh ra mà miền không phụ thuộc trình biên dịch	Port / Adapter	`ICodeAnalyzer` (miền) ← `TypeScriptCodeAnalyzer` (hạ tầng)
Quy tắc chấm điểm chất lượng mã	Policy / Specification	`CodeQualityPolicy`, `LayerRules`



Hình 4.2. *Mẫu Strategy cho tác tử*

4.3 Thiết kế tác tử

4.3.1 Template Method và hai chiến lược

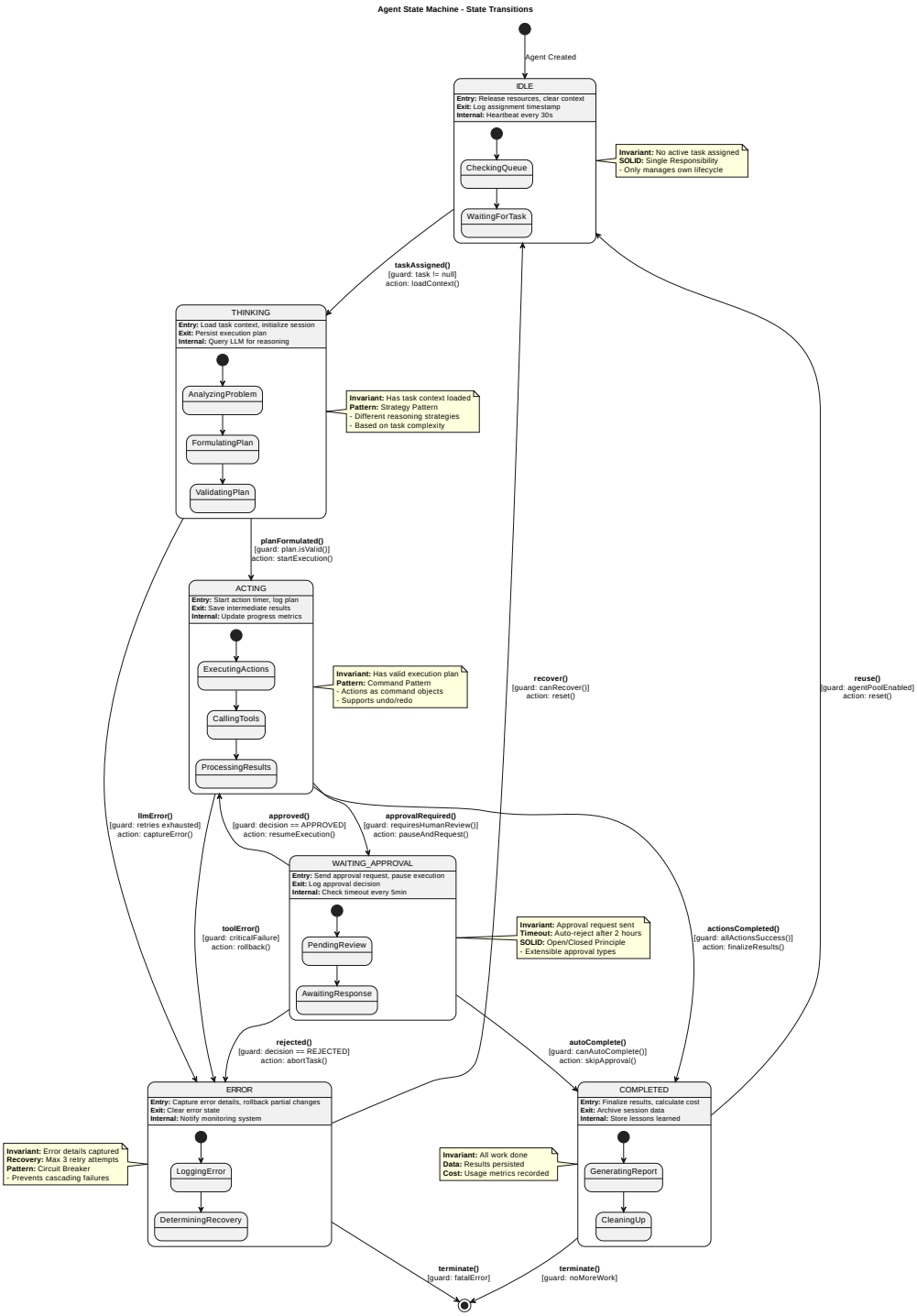
Mỗi tác tử là một đối tượng `BaseAgent` được cấu hình bằng ba thứ: một `RoleProfile` (vai trò, các loại công việc hỗ trợ, loại công việc mặc định, lời nhắc hệ thống), một chiến lược tư duy và một chiến lược hành động. `BaseAgent.execute()` cố định khung thuật toán:

1. Chuẩn hoá ngữ cảnh: loại công việc được đối chiếu với hồ sơ vai trò, nên chiến lược luôn nhận một loại công việc vai trò đó hỗ trợ.
2. Phát `TaskAssignedEvent`.
3. Chuyển `IDLE → THINKING`, gọi `thinkingStrategy.think(context)`: nạp bộ nhớ và bài học liên quan, gọi LLM để lập kế hoạch.
4. Chuyển `THINKING → ACTING`, gọi `actionStrategy.act(context, thought)`: sinh tài liệu hoặc mã, ghi vào kho tệp của dự án, chạy công cụ.
5. Chuyển `ACTING → COMPLETED → IDLE`, lưu ký ức, phát `AgentCompletedTaskEvent`.
6. Khi có ngoại lệ: chuyển sang `ERROR`; lỗi tạm thời cho phép quay lại `IDLE`.

Thiết kế này thoả SRP (tác tử chỉ điều phối vòng đời, chiến lược chứa chuyên môn), OCP (thêm vai trò là thêm hồ sơ và hai chiến lược) và LSP (mọi tác tử thay thế được cho nhau qua `IAgent`).

4.3.2 Máy trạng thái theo từng lần thực thi

Ban đầu mỗi vai trò giữ một máy trạng thái duy nhất. Kiểm thử phát hiện hai hậu quả: hai công việc cùng vai trò chạy đồng thời làm hỏng nhau (“Invalid state transition: IDLE → ERROR”), và sau một lỗi vĩnh viễn vai trò kẹt ở `ERROR`, từ chối mọi công việc sau đó. Thiết kế đã sửa: **mỗi lần `execute()` có một máy trạng thái riêng**, tác tử giữ tập các máy đang chạy và báo trạng thái của lần gần nhất. Đây là một ví dụ về việc mẫu State phải gắn với đúng đối tượng có vòng đời (lần thực thi), không phải đối tượng tồn tại lâu (tác tử).



Hình 4.3. *Máy trạng thái của tác tử*

4.3.3 Gọi LLM và ghi chi phí

Chiến lược không gọi nhà cung cấp LLM trực tiếp. `AgentLLMService` (Facade) tra cấu hình mô hình của vai trò, lấy nhà cung cấp từ `LLMProviderFactory` và bọc nó bằng `UsageTrackingLLMProvider` (Decorator). Decorator chuyển nguyên lời gọi rồi ghi số token và chi phí vào `ApiUsageLog`, cộng vào chi phí của dự án, và phát

`BudgetExceededEvent` một lần khi vượt ngân sách. Nhờ vậy chi phí của cả tám vai trò được ghi đầy đủ mà không chiến lược nào phải biết về chi phí.

4.3.4 Học từ kết quả

Cơ chế học gồm ba phần:

1. `OutcomeSignalPolicy` (chính sách thuần) quyết định tín hiệu nào chấm lần thực thi nào: kết quả cổng OOAD chấm BA và Architect; kết quả của QC (kiểm thử đạt, use case thiếu) chấm DEV; phán quyết QA chấm DEV và QC.
2. `LearningService` lưu mỗi kết quả vào bộ nhớ của vai trò; cứ mỗi N kết quả (mặc định 3) nó tinh luyện các kết quả gần đây thành một ký ức `lessons_learned` bằng một lời gọi LLM, được tính chi phí như mọi lời gọi khác.
3. Chiến lược tư duy nạp các bài học trước tiên khi xây lời nhắc.

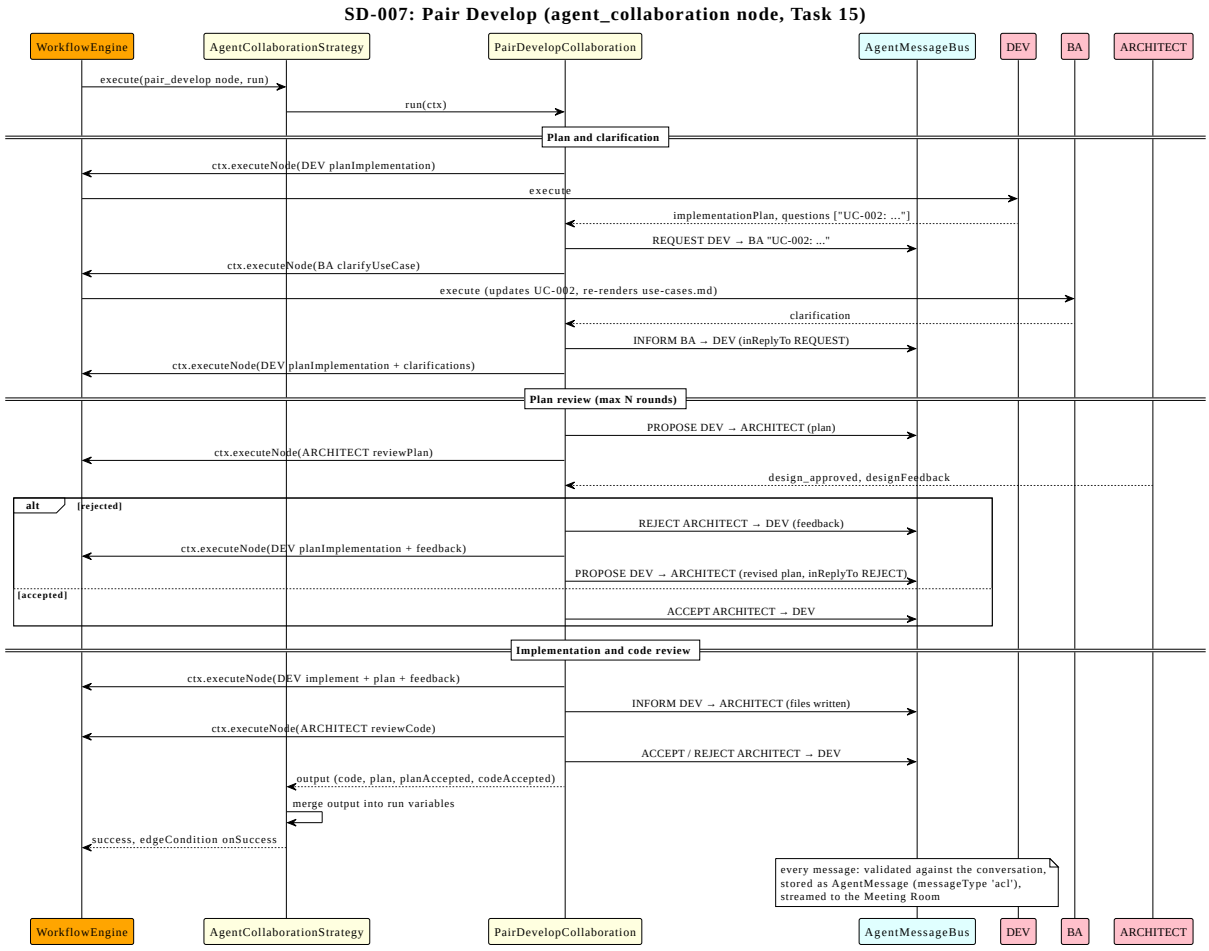
Thiết kế này gần với Reflexion [28] ở chỗ phản hồi bằng ngôn ngữ, nhưng tín hiệu là khách quan (cổng, kiểm thử, phán quyết) chứ không do chính tác tử tự đánh giá.

4.3.5 Cộng tác giữa các tác tử

Ba mẫu cộng tác là các chiến lược của một nút quy trình, đăng ký trong `CollaborationRegistry`:

- **Lập trình cặp** (`pair_develop`): DEV lập kế hoạch; câu hỏi về use case mơ hồ được gửi BA trước (vòng làm rõ); DEV gửi `PROPOSE`, Architect `ACCEPT` hoặc `REJECT` kèm phản hồi; sau khi hiện thực, Architect rà soát mã. Số vòng có giới hạn để chi phí bị chặn.
- **Vòng làm rõ** (`clarification_loop`): tác tử hỏi BA và nhận câu trả lời trước khi tiếp tục.
- **Chuỗi rà soát** (`review_chain`, Chain of Responsibility): QC rồi QA; một mắt xích từ chối thì dừng chuỗi.

Mọi lượt trao đổi đi qua `AgentMessageBus` (Mediator) với các performative theo tinh thần FIPA-ACL [16], và được hiển thị trong Phòng họp (Meeting Room) của giao diện.



Hình 4.4. *Cộng tác lập trình cặp có vòng làm rõ*

4.4 Thiết kế bộ máy quy trình

4.4.1 Định nghĩa và thực thi

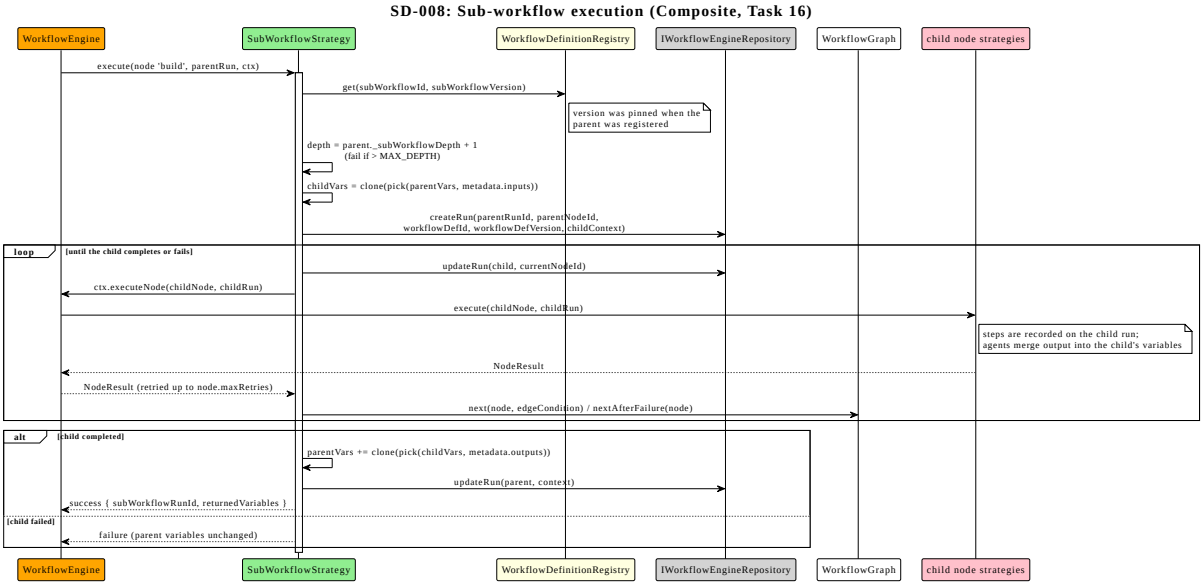
Một quy trình là một đồ thị gồm các nút có kiểu và các cạnh (mặc định, có điều kiện, `onFail`). `WorkflowEngine` không chứa logic của từng loại nút: mỗi loại là một `NodeExecutionStrategy` tự đăng ký với bộ máy. Thêm loại nút (ví dụ cổng chất lượng) là thêm một lớp, không sửa bộ máy (OCP).

Quy trình mặc định cho một dự án phần mềm:

```
CEO (mốc)
 → [quy trình con: phân tích & thiết kê]  BA → Architect → cổng OOAD —không đạt→
Architect
 → PM (một công việc cho mỗi use case)
 → [quy trình con: xây dựng & kiểm chứng] DEV → QC → QA → [điều kiện qa_approved]
—không đạt→ DEV
 → Marketing → [phê duyệt của con người] → CEO (đóng dự án)
```

4.4.2 Quy trình con, phiên bản và khôi phục

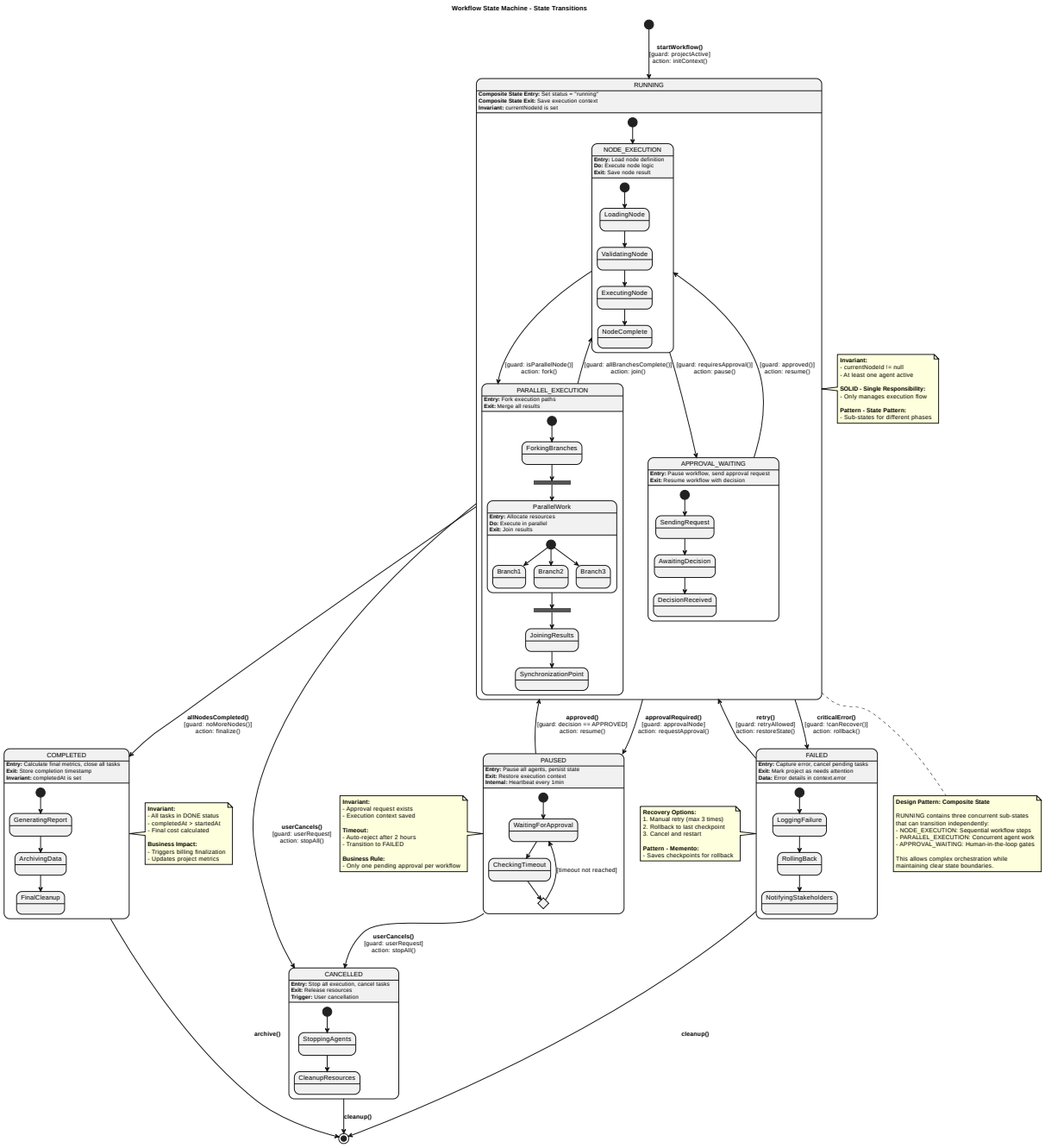
- **Quy trình con (Composite):** nút `sub_workflow` chạy một quy trình khác như một lượt chạy con có `parentRunId`; kết quả của lượt con được gộp vào ngữ cảnh lượt cha. Ba mẫu quy trình có kiểu (dự án phần mềm OOAD, backend REST API, chỉ phân tích và thiết kế) dùng chung các quy trình con.
- **Phiên bản bất biến:** mỗi định nghĩa được lưu theo `id@phiên_bản` cùng mã băm của nội dung chuẩn hoá; đăng ký lại cùng phiên bản với nội dung khác bị từ chối. Mỗi lượt chạy ghim một phiên bản cụ thể.
- **Ảnh chụp và khôi phục (Memento):** sau mỗi nút, `WorkflowSnapshotCaretaker` lưu một memento gồm nút hiện tại, ngữ cảnh và trạng thái; người dùng có thể khôi phục một lượt chạy đang tạm dừng về một ảnh chụp, các phê duyệt treo sau ảnh chụp đó bị huỷ.
- **Song song có phụ thuộc:** nút `parallel` chạy các nhánh theo thứ tự tô-pô của `dependsOn` với giới hạn đồng thời; `DependencyResolver` phát hiện chu trình.



Hình 4.5. *Thực thi quy trình con*

4.4.3 Xử lý lỗi

Một nút thất bại được thử lại tối đa ba lần với giãn cách mũ. Nếu vẫn thất bại, lượt chạy đi theo cạnh `onFail` nếu có, ngược lại chuyển sang `failed` và phát `workflow.failed`. Hai lỗi thiết kế đã được kiểm thử phát hiện và sửa: nút thất bại từng để lượt chạy BullMQ ở trạng thái `running` mãi mãi, và từng đi tiếp theo cạnh thành công.



Hình 4.6. *Máy trạng thái của quy trình*

4.5 Thiết kế dữ liệu và lưu trữ

Dữ liệu có cấu trúc nằm trong PostgreSQL qua 20 mô hình Prisma (người dùng, dự án, mốc, công việc, tác tử, bộ nhớ, lượt chạy, bước, ảnh chụp, phiên bản định nghĩa, phê duyệt, nhật ký sử dụng API, nhật ký cổng chất lượng, nhật ký kiểm toán, khoá API, webhook, lần gửi webhook, dự báo…). Lược đồ được quản lý bằng migration có phiên bản; một bước kiểm tra so lược đồ dựng từ migration với mô hình để phát hiện lệch.

Sản phẩm của tác tử (tài liệu, sơ đồ, mã, kiểm thử, báo cáo) **không** nằm trong cơ sở dữ liệu mà trong `ProjectWorkspace`: một thư mục bền vững cho mỗi dự án, có kiểm tra chống thoát thư mục (đường dẫn `../../etc/...` do LLM đề xuất bị từ chối). Thiết kế ban đầu ghi sản phẩm vào sandbox Docker, vốn chạy mỗi lệnh trong một container mới không có ổ gắn, nên mọi tệp bị mất sau mỗi lệnh; đây là lý do cổng chất lượng và việc quét kiểm thử trước đó không bao giờ thành công.

4.6 Áp dụng OOAD cho dự án sinh ra

Đây là đóng góp đặc thù của nền tảng: mọi dự án đi qua cùng một chuỗi OOAD và chuỗi đó được **kiểm tra**, không chỉ được sinh ra.

4.6.1 Chuỗi sản phẩm

Bước	Tác tử	Sản phẩm trong kho tệp của dự án
Yêu cầu	BA	`docs/use-cases.md`
Mô hình miền và UML	Architect	`docs/domain-model.puml`, `docs/use-case-diagram.puml`, `docs/class-diagram.puml`, `docs/sequence-UC-xxx.puml`, `docs/architecture-patterns.md`, `ARCHITECTURE.md`
Khung mã	Architect	`src/domain/{entities,value-objects,repositories}`, `src/application/services`, `src/infrastructure/persistence`, `src/presentation/controllers`, `src/composition-root.ts`
Hiện thực	DEV	Mã đầy đủ trong khung, chú thích `UC-xxx`
Chất lượng mã	Hệ thống	`docs/code-quality.md`
Kiểm thử	QC	`tests/UC-xxx.spec.ts`, kết quả Jest
Truy vết	QC	`docs/traceability-matrix.md`

Khung mã dựng từ mẫu Handlebars theo đúng bốn tầng; giao diện Repository nằm trong miền, hiện thực trong hạ tầng, và một `composition-root.ts` lắp ghép phụ thuộc. Như vậy kiến trúc mà nền tảng dùng cho chính nó cũng là kiến trúc nó đòi hỏi ở sản phẩm.

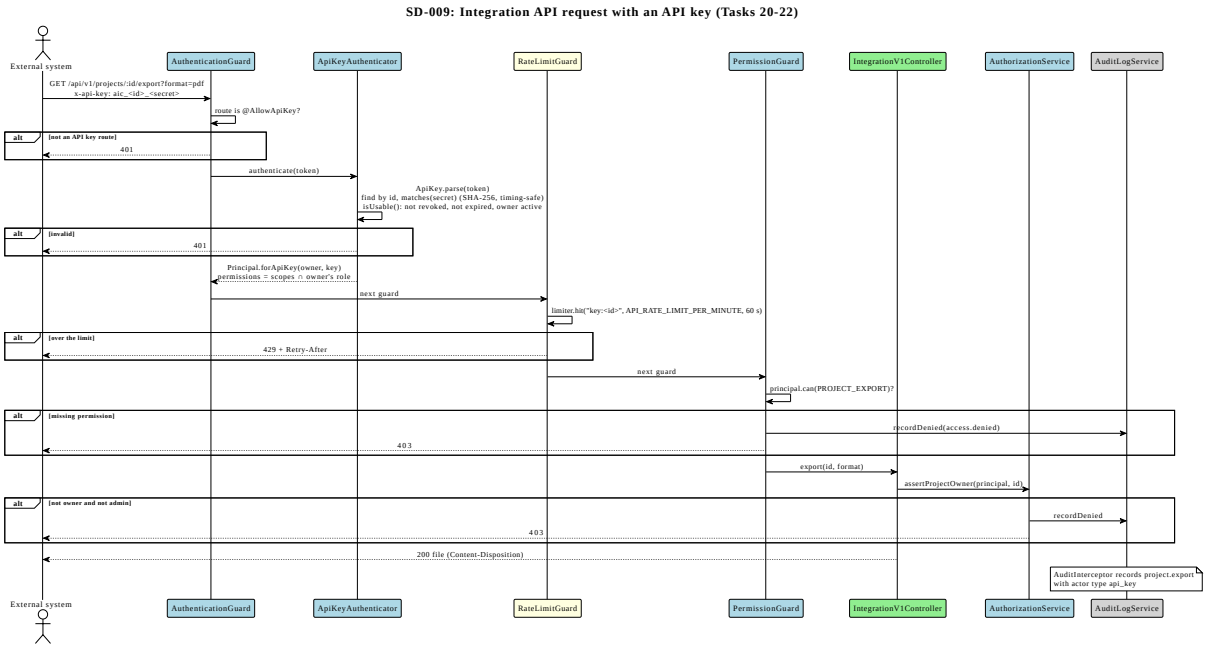
4.6.2 Ba lớp kiểm tra

1. **Cổng OOAD** (nút `quality_gate` sau thiết kế): kiểm tra số use case, số thực thể miền, số mẫu thiết kế đạt ngưỡng và các tệp bắt buộc tồn tại. Không đạt thì quay lại Architect. Mỗi lần đánh giá ghi vào `QualityGateLog` và phát `QualityGateEvaluatedEvent`.

2. **Phân tích tĩnh trước QA** (mục 4.8.3): trong đó có luật phân lớp *miền ← ứng dụng ← hạ tầng/trình bày*; một lệnh `import` hoặc `require()` từ miền vào hạ tầng là vi phạm.
3. **Kiểm thử chấp nhận đã thực thi**: độ bao phủ use case chỉ tính các use case có kiểm thử chạy đạt; use case thiếu kiểm thử hoặc kiểm thử thất bại được liệt kê đích danh.

4.7 Bảo mật và tính năng doanh nghiệp

- **Chuỗi guard toàn cục** (Chain of Responsibility): `AuthenticationGuard` (JWT [49] cho người dùng, khoá API cho `/api/v1`) → `RateLimitGuard` → `PermissionGuard`. Mặc định mọi route cần đăng nhập; route công khai phải đánh dấu `@Public`.
- **Quyền sở hữu**: `AuthorizationService` kiểm tra chủ dự án và người được giao phê duyệt; vi phạm trả 403 và ghi `access.denied`.
- **Nhật ký kiểm toán chỉ ghi thêm**: decorator `@Audited` cùng `AuditInterceptor` ghi hành động của người dùng; `AuditLogHandler` ghi hành động của hệ thống và tác tử từ sự kiện miền. Mật khẩu, token, bí mật và khoá bị che trước khi ghi.
- **API tích hợp**: khoá API chỉ mang các phạm vi mà vai trò hiện tại của chủ khoá cho phép (giao của hai tập); giới hạn tần suất theo khoá; tài liệu OpenAPI ở `/api/docs` (có thể tắt trong production).
- **Webhook**: payload ký HMAC-SHA256 [48] kèm dấu thời gian, URL tới mạng nội bộ bị từ chối cả sau khi phân giải DNS (chống SSRF [50]), gửi lại với giãn cách mũ.
- **Phê duyệt chỉ qua REST**: kiểm thử đầu-cuối của cấu hình triển khai phát hiện rằng giao diện gửi quyết định phê duyệt qua WebSocket, và cổng WebSocket chấp nhận quyết định từ bất kỳ kết nối nào: người xem, vốn nhận 403 từ REST, vẫn duyệt được và dự án được đóng mà không có mục kiểm toán. Thiết kế đã sửa: quyết định chỉ đi qua `POST /api/approvals/:id/approve|reject` (kiểm quyền, kiểm người được giao, ghi kiểm toán), thông điệp WebSocket tương ứng bị từ chối.
- **Cấu hình production an toàn**: với `NODE_ENV=production`, API không khởi động nếu thiếu `JWT_SECRET` (hoặc ngắn hơn 32 ký tự), thiếu `CORS_ORIGIN` hoặc còn giá trị giữ chỗ như `CHANGE_ME`. Mọi nơi đọc khoá JWT (REST, guard WebSocket, cổng WebSocket) dùng chung một hàm; trước đó ba nơi có ba giá trị dự phòng khác nhau, nên khi không đặt `JWT_SECRET` kết nối thời gian thực luôn bị từ chối.



Hình 4.7. *Chuỗi xử lý một yêu cầu có khoá API*

4.8 Phân tích, dự báo và chất lượng mã

4.8.1 Phân tích hiệu năng tác tử

`AgentPerformanceAnalyzer` (miền, thuần) tính cho mỗi vai trò: số bước, tỉ lệ thành công, thời lượng trung bình và p95, chi phí trên mỗi lần thành công, chất lượng, xu hướng so với kỳ trước và thứ hạng. Điểm thiết kế quan trọng là **chất lượng được đo theo vai trò** bằng các `IQualitySignal` khác nhau: DEV theo tỉ lệ được QA duyệt, QC theo kết quả kiểm thử, Architect theo cổng OOAD, các vai trò khác theo tỉ lệ thành công ngay lần đầu. `AgentAnomalyDetector` áp dụng năm quy tắc (tỉ lệ thành công giảm 20 điểm, ba lần thất bại liên tiếp, chi phí gấp đôi, chậm gấp đôi, chất lượng dưới 50%); mỗi bất thường được phát một lần mỗi ngày qua cùng đường sự kiện, kiểm toán và webhook.

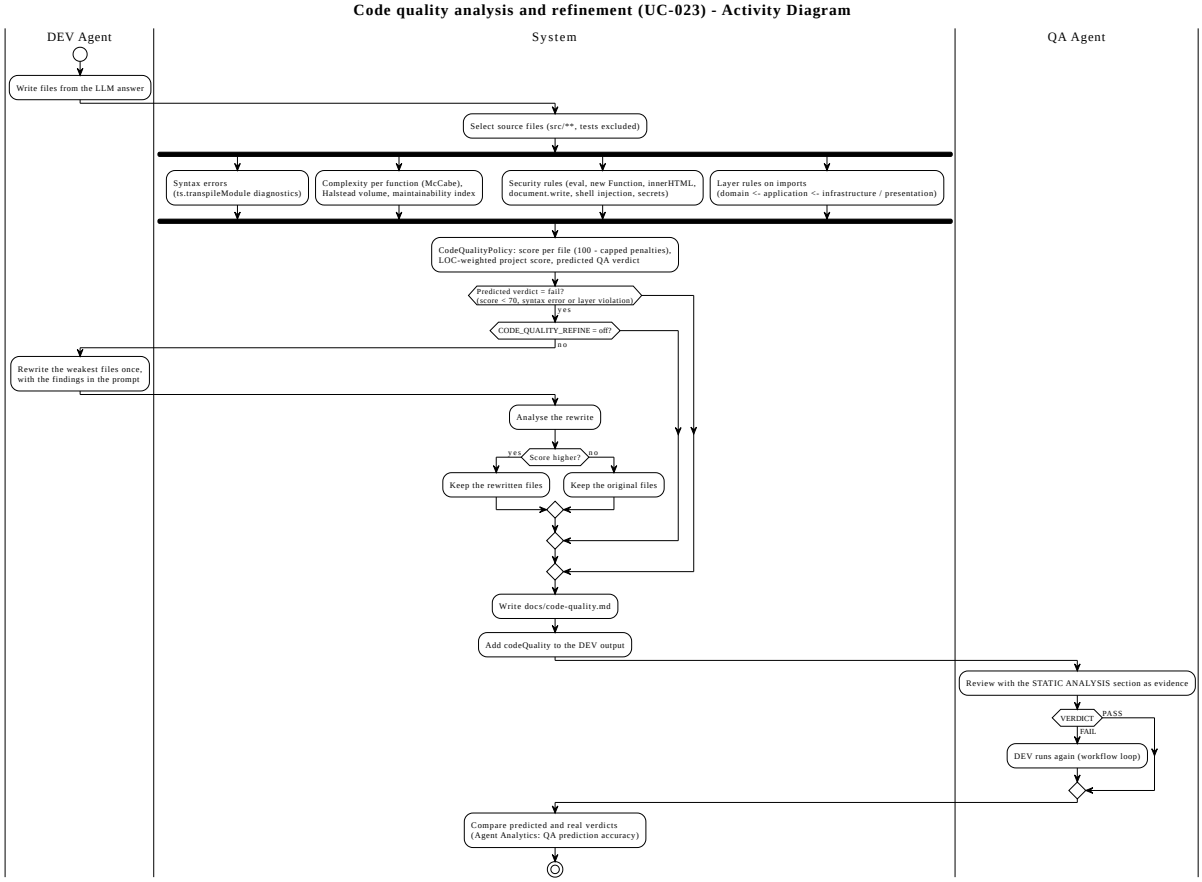
4.8.2 Dự báo thời gian và chi phí

Nền tảng không ghi giờ công của con người; cái nó ghi là thời lượng và chi phí LLM của từng bước tác tử. Vì vậy mô hình dự báo **thời gian tác tử và chi phí LLM theo dự án và vai trò**, không phải giờ công của từng công việc. Mỗi vai trò có một hồi quy ridge [40] trên hai đặc trưng (độ dài mô tả, số use case); khi một vai trò có ít hơn 6 mẫu, mô hình dùng trung vị. Độ chính xác được báo bằng sai số leave-one-out (MAPE và MAE) [46], và mỗi dự báo mang mức tin cậy. Dự báo được lưu khi quy trình bắt

đầu và khi PM lập kế hoạch, rồi được so với kết quả khi dự án xong. Chi phí ước lượng của mỗi công việc mà PM tạo được suy ra từ dự báo của DEV, QC và QA.

4.8.3 Phân tích tĩnh và vòng sửa có giới hạn

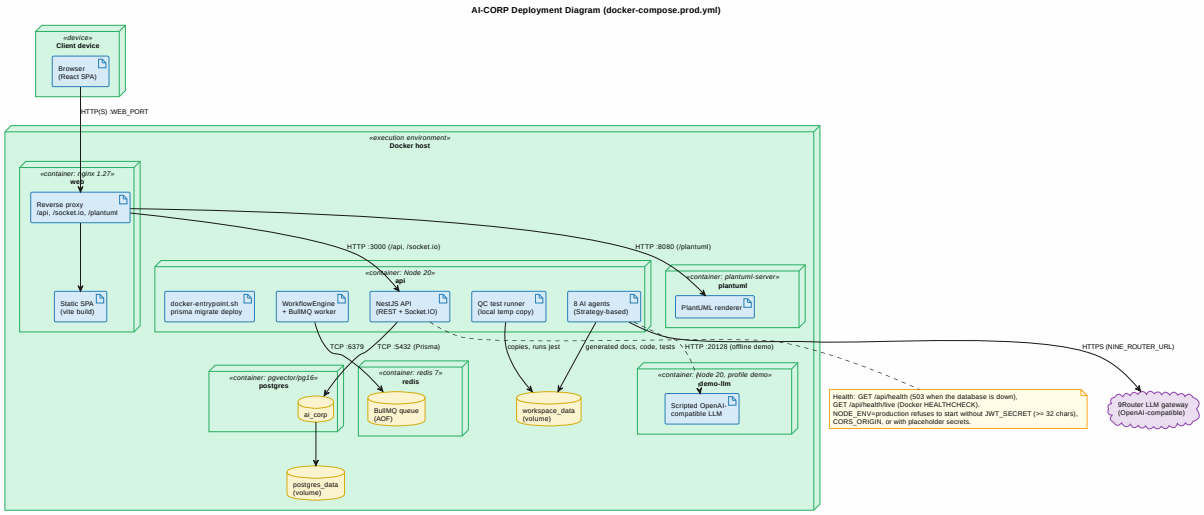
`TypeScriptCodeAnalyzer` (hạ tầng, sau cổng `ICodeAnalyzer`) dùng TypeScript compiler API để tính: lỗi cú pháp, độ phức tạp McCabe từng hàm [35], thể tích Halstead [36], chỉ số bảo trì chuẩn hoá [37], [38], lời gọi không an toàn (`eval`, `Function`), bí mật viết cứng, vi phạm phân lớp và tham chiếu use case. `CodeQualityPolicy` (miền) trừ điểm có trần cho mỗi loại vấn đề, tính điểm trung bình có trọng số theo số dòng, và **dự đoán phán quyết QA** là *fail* khi điểm dưới 70, có lỗi cú pháp hoặc có vi phạm phân lớp. Khi dự đoán là *fail*, DEV được cho **đúng một** vòng sửa kèm danh sách vấn đề; bản sửa chỉ được giữ nếu điểm tăng. QA nhận báo cáo phân tích làm bằng chứng khi rà soát. Dự đoán được đối chiếu với phán quyết thật để đo độ chính xác thay vì được khẳng định.



Hình 4.8. *Luồng phân tích chất lượng mã*

4.9 Thiết kế triển khai

Hệ thống triển khai bằng Docker Compose trên một máy chủ: nginx phục vụ ứng dụng web và làm proxy ngược cho `/api`, `/socket.io` và `/plantuml`; API (Node 20) chạy migration khi khởi động rồi phục vụ; PostgreSQL (có pgvector), Redis (AOF) và máy chủ PlantUML nội bộ; kho tệp của dự án là một volume. Chỉ nginx được công bố ra ngoài. Một hồ sơ `demo` thêm LLM kịch bản để trình diễn ngoại tuyến. API có hai điểm kiểm tra sức khoẻ [54]: `/api/health/live` (tiến trình còn trả lời, dùng cho HEALTHCHECK của Docker) và `/api/health` (sẵn sàng: trả 503 khi mất cơ sở dữ liệu, *degraded* khi mất Redis hoặc cổng LLM).



Hình 4.9. *Sơ đồ triển khai*

Chương 5
HIỆN THỰC

Chương này mô tả cách thiết kế ở chương 4 được hiện thực: công nghệ, quy mô mã, các đoạn mã tiêu biểu cho từng mẫu chính, giao diện người dùng và cấu hình triển khai.

5.1 Công nghệ và quy mô

Thành phần	Công nghệ
Ngôn ngữ	TypeScript 5 (API, web, dự án sinh ra)
API	NestJS 10, Socket.IO, Prisma 5, BullMQ 5, pdfkit, TypeScript compiler API
Cơ sở dữ liệu, hàng đợi	PostgreSQL 16 (pgvector), Redis 7
Web	React 18, Vite, Ant Design, TanStack Query, Zustand, Recharts
Kiểm thử	Jest + ts-jest (API), Vitest (web), Playwright (giao diện đầu-cuối)
LLM	Cổng bất kỳ tương thích OpenAI (`NINE_ROUTER_URL`), nhà cung cấp Anthropic trực tiếp tuỳ chọn
Triển khai	Docker (Node 20, nginx 1.27), Docker Compose
Tổ chức mã	Monorepo pnpm + Turborepo: `apps/api`, `apps/web`, `packages/shared-types`

Quy mô tại thời điểm viết (đếm tự động, không tính tệp kiểm thử):

Phần	Tệp	Dòng
API (`apps/api/src`)	326	27 336
trong đó tầng miền (`src/domain`)	101	6 893
Web (`apps/web/src`)	70	7 671
Tệp kiểm thử API	47	—

API có 26 mô-đun NestJS và 277 lớp được xuất (`export class`).

5.2 Tầng miền

Tầng miền chia theo khái niệm: `entities` (Project, Task, Milestone, Agent…), `value-objects` (Money, ProjectStatus…), `repositories` (giao diện), `events` (sự kiện miền và `EventBus`), `agent` (máy trạng thái, bộ phát sự kiện, giao diện chiến lược, học), `workflow` (đồ thị, bộ giải phụ thuộc, phiên bản ngữ nghĩa, memento), `security`, `audit`, `integration`, `reporting`, `analytics`, `estimation`, `quality`, `collaboration`.

Thực thể bảo vệ bất biến của mình. Ví dụ, `Task.transitionTo` chỉ cho phép các chuyển trạng thái hợp lệ (todo → in_progress → review → done, review → in_progress khi QA từ chối), và `Money` không cho giá trị âm. Kiểm thử miền từng phát hiện `Task.getCostVariance()` ném ngoại lệ với mọi công việc dưới ngân sách vì chênh lệch âm không biểu diễn được bằng `Money`; hàm đã được sửa để trả về số đô-la có dấu.

```
        // apps/api/src/domain/repositories/IProjectRepository.ts (rút gọn)
        export interface IProjectRepository {
          findById(id: string): Promise<ProjectDTO | null>;
          findAll(options?: { orderBy?: 'createdAt' | 'updatedAt'; order?: 'asc'
| 'desc' }): Promise<ProjectDTO[]>;
          create(data: CreateProjectData): Promise<ProjectDTO>;
          update(id: string, data: UpdateProjectData): Promise<ProjectDTO>;
        }
```

Hiện thực `PrismaProjectRepository` nằm ở `src/infrastructure/persistence` và chỉ chọn các cột mà DTO cần; kiểm thử tải từng phát hiện danh sách dự án tải cả ngữ cảnh JSON của lượt chạy mới nhất rồi bỏ đi (46 → 9 ms sau khi sửa).

5.3 Tác tử

```
        // apps/api/src/modules/agents/base.agent.ts (rút gọn)
        async execute(input: AgentContext): Promise<AgentResult<TOutput>> {
          const context = this.prepareContext(input);          // loại công việc
theo RoleProfile
          const lifecycle = new AgentStateMachine(AgentState.IDLE); // một máy
trạng thái mỗi lần chạy
          this.running.add(lifecycle);
          try {
            await this.publish(new TaskAssignedEvent(/* … */));
            this.transition(lifecycle, AgentState.THINKING, 'Starting thought
process');
            const thought = await this.thinkingStrategy.think(context);   //
Strategy
            this.transition(lifecycle, AgentState.ACTING, 'Executing action');
            const action = await this.actionStrategy.act(context, thought); //
Strategy
            this.transition(lifecycle, AgentState.COMPLETED, 'Done');
            // lưu ký ức, phát AgentCompletedTaskEvent, quay về IDLE …
          } catch (error) {
            this.transition(lifecycle, AgentState.ERROR, (error as
Error).message);
            // …
          } finally {
            this.running.delete(lifecycle);
          }
        }
```

Mỗi vai trò có một tệp hồ sơ trong `modules/agents/profiles` và hai chiến lược trong `modules/agents/strategies`. Danh sách tác tử được NestJS tiêm qua token `AGENTS`; `AgentOrchestrator` dựng một bảng tra theo vai trò và báo lỗi nếu hai tác tử trùng vai trò. Như vậy, thêm một vai trò thứ chín gồm: thêm giá trị `AgentRole`, một hồ sơ, hai chiến lược và một dòng đăng ký nhà cung cấp; bộ điều phối và bộ máy quy trình không đổi.

Các chiến lược hành động của BA, Architect, DEV và QC ghi sản phẩm qua `ProjectWorkspace`. `OoadArtifactService` (Facade) dựng tài liệu và khung mã từ mẫu Handlebars; nội dung do LLM trả về được tách thành các khối `<CODE filepath="...">` và từng đường dẫn được kiểm tra trước khi ghi.

5.4 Bộ máy quy trình

`WorkflowEngine` giữ một bảng `type → NodeExecutionStrategy`. Khi có Redis, mỗi nút là một công việc BullMQ (đồng thời tối đa 5); khi không, nút được thực thi trực tiếp. Kết thúc mỗi nút, `handleJobCompletion` ghi bước với thời lượng, chụp memento, chọn

cạnh kế tiếp; khi nút thất bại sau các lần thử, kết quả được chuyển thành `failure` để lượt chạy đi theo `onFail` hoặc kết thúc ở `failed`.

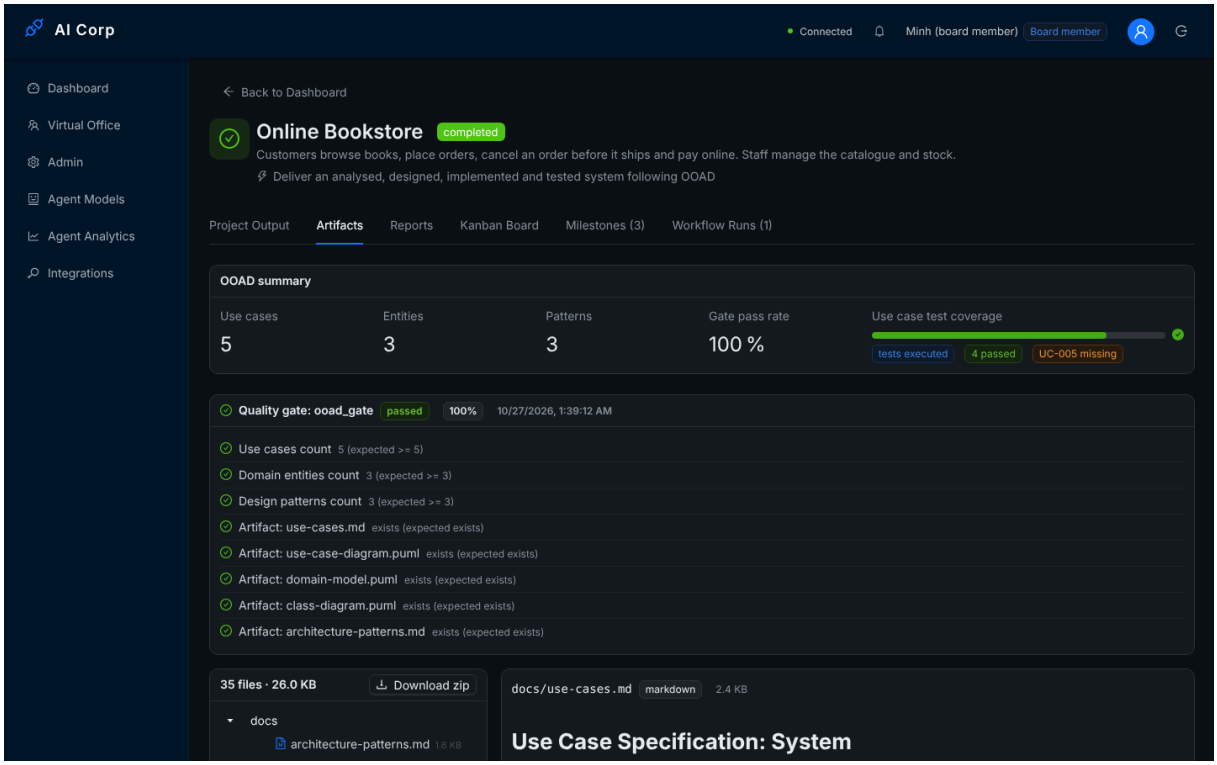
Các mẫu quy trình là lớp con của `WorkflowTemplate` với danh sách tham số có kiểu (số, chuỗi, enum, boolean) được kiểm tra trước khi dựng định nghĩa. Ví dụ, mẫu *backend REST API* có tham số `database` kiểu enum; giá trị được truyền vào lời nhắc thiết kế của Architect như một ràng buộc của nút, nên các Repository được thiết kế cho đúng cơ sở dữ liệu đã chọn.

5.5 Tính năng doanh nghiệp

- **Phân quyền:** `@RequirePermission(Permission.PROJECT_CREATE)` trên controller; quyền của vai trò lấy từ `shared-types` để web và API dùng chung một định nghĩa. Giao diện ẩn hành động người dùng không có quyền (ví dụ nút *New Project* với người xem) nhưng API vẫn kiểm tra độc lập.
- **Kiểm toán:** `@Audited({ action: 'approval.approve', resourceType: 'Approval', idParam: 'id' })` trên route; interceptor ghi kết quả *ok* hoặc *denied* kèm IP; nhật ký có bộ lọc, phân trang và xuất CSV (ô bắt đầu bằng `=`, `+`, `-`, `@` được thoát để chống chèn công thức).
- **Xuất báo cáo:** JSON, CSV và PDF (pdfkit, phông có dấu tiếng Việt); báo cáo tuần được ghi vào thư mục `reports/` của dự án thay vì gửi email.

5.6 Giao diện người dùng

Ứng dụng web (nhãn giao diện bằng tiếng Anh) có các màn hình: Bảng điều khiển (danh sách dự án, chi phí), Văn phòng ảo (trạng thái tám tác tử theo thời gian thực và Phòng họp), Chi tiết dự án (kết quả, Sản phẩm, Báo cáo, Kanban, Mốc, Lượt chạy), Phân tích tác tử, Người dùng và vai trò, Nhật ký kiểm toán, Tích hợp. Tab *Artifacts* (Sản phẩm) hiển thị tóm tắt OOAD, kết quả cổng chất lượng, cây tệp của dự án, trình xem Markdown, mã và sơ đồ PlantUML (render bởi máy chủ PlantUML nội bộ), và nút tải toàn bộ dự án dạng zip.



Hình 5.1. *Tab Sản phẩm của dự án Online Bookstore*

Trong lúc chuẩn bị trình diễn, hai lỗi hiển thị được phát hiện và sửa: thẻ dự án luôn ghi “Starting… 0%” vì danh sách dự án không kèm lượt chạy (nay dùng trạng thái dự án khi không có lượt chạy, có kiểm thử đơn vị), và tổng chi phí hiển thị số thực chưa làm tròn.

5.7 Cấu hình triển khai

- `apps/api/Dockerfile`: hai giai đoạn; giai đoạn build cài phụ thuộc, sinh Prisma client, build API và script seed; giai đoạn chạy dùng `tini`, chạy bằng người dùng `node`, có phông DejaVu cho PDF và HEALTHCHECK gọi `/api/health/live`. `docker-entrypoint.sh` chạy `prisma migrate deploy` rồi khởi động.
- `apps/web/Dockerfile`: build với `VITE_API_URL=/api`, `VITE_WS_URL=/`, `VITE_PLANTUML_SERVER=/plantuml` để mọi lời gọi cùng nguồn gốc qua nginx.
- `docker-compose.prod.yml` và `.env.production.example`: sáu dịch vụ (một chỉ chạy ở hồ sơ demo), ba volume, chỉ công bố cổng web.

Việc chuẩn bị triển khai phát hiện bốn lỗi khiến cấu hình cũ không thể chạy: thư mục migration chỉ có lược đồ ban đầu (thiếu 8 bảng và nhiều cột, vì quá trình phát triển dùng `prisma db push`) và tệp khoá migration nằm sai chỗ; cấu hình nginx bỏ tiền tố `/api` nên mọi lời gọi API trả 404 và không chuyển tiếp WebSocket; Dockerfile của API

gọi health ở đường dẫn sai và chạy tệp không tồn tại. Cả bốn đã được sửa và kiểm chứng (mục 6.3).

Chương 6
KIỂM THỬ VÀ ĐÁNH GIÁ

Chương này trình bày cách nền tảng được kiểm chứng và các kết quả đo được. Mọi số liệu lấy từ các lần chạy có lưu vết trong kho mã; lệnh tái tạo nằm ở Phụ lục C và D.

6.1 Chiến lược kiểm thử

Kiểm thử được tổ chức theo kim tự tháp kiểm thử [52]:

1. **Kiểm thử đơn vị** cho tầng miền (hàm thuần, không giả lập) và cho dịch vụ ứng dụng (với Repository trong bộ nhớ, LLM giả lập, sandbox giả lập).
2. **Kiểm thử tích hợp HTTP**: một ứng dụng Nest thật với các guard, interceptor và controller thật, dữ liệu trong bộ nhớ, gọi qua HTTP.
3. **Kiểm thử đầu-cuối** trên hạ tầng thật: API đã build, PostgreSQL, Redis, nginx, một LLM kịch bản tương thích OpenAI, trình duyệt Playwright cho giao diện. Mỗi lần chạy có một tệp điều khiển in một dòng cho mỗi kiểm tra và trả mã lỗi khác 0 nếu có kiểm tra thất bại.
4. **Kiểm thử tải** có ngân sách cho từng endpoint.

LLM kịch bản là lựa chọn có chủ đích: nó trả lời theo nội dung lời nhắc (ví dụ dự án tên "Legacy…" nhận mã lần đầu vi phạm phân lớp), có độ trễ tăng theo độ dài lời nhắc và một tên mô hình có giá, nên thời lượng và chi phí thay đổi giữa các dự án, đủ để huấn luyện mô hình dự báo, trong khi mọi lần chạy cho cùng kết quả.

6.2 Kiểm thử tự động và độ bao phủ

Bộ kiểm thử	**Kết quả**
API (Jest)	47 bộ, **674 đạt**, 15 bỏ qua (cần Docker daemon cho sandbox), 0 thất bại
Web (Vitest)	41 đạt
Biên dịch	`nest build` 0 lỗi; `tsc --noEmit` của web 0 lỗi; `vite build` thành công

Ngưỡng độ bao phủ được đặt trong cấu hình Jest, nên lệnh `npm run test:cov` thất bại nếu không đạt:

Phạm vi	Dòng	Lệnh	Hàm	Nhánh	Ngưỡng (dòng/lệnh/hàm/nhánh)
Tầng miền (`src/domain`)	**97,1%**	**96,6%**	**97,2%**	**89,6%**	90 / 90 / 90 / 85
Toàn API	77,0%	77,2%	75,2%	63,7%	sàn 70 / 70 / 64 / 55

Lý do đặt 90% trên tầng miền thay vì toàn bộ: tầng miền chứa các quy tắc nghiệp vụ, còn controller và phần nối NestJS chủ yếu là mã khung; ép 90% trên chúng sẽ dẫn tới kiểm thử chính framework. Toàn API vẫn có ngưỡng sàn để độ bao phủ không giảm.

6.3 Kiểm thử đầu-cuối

Ba lần chạy đầu-cuối chính, mỗi lần trên một cơ sở dữ liệu trống:

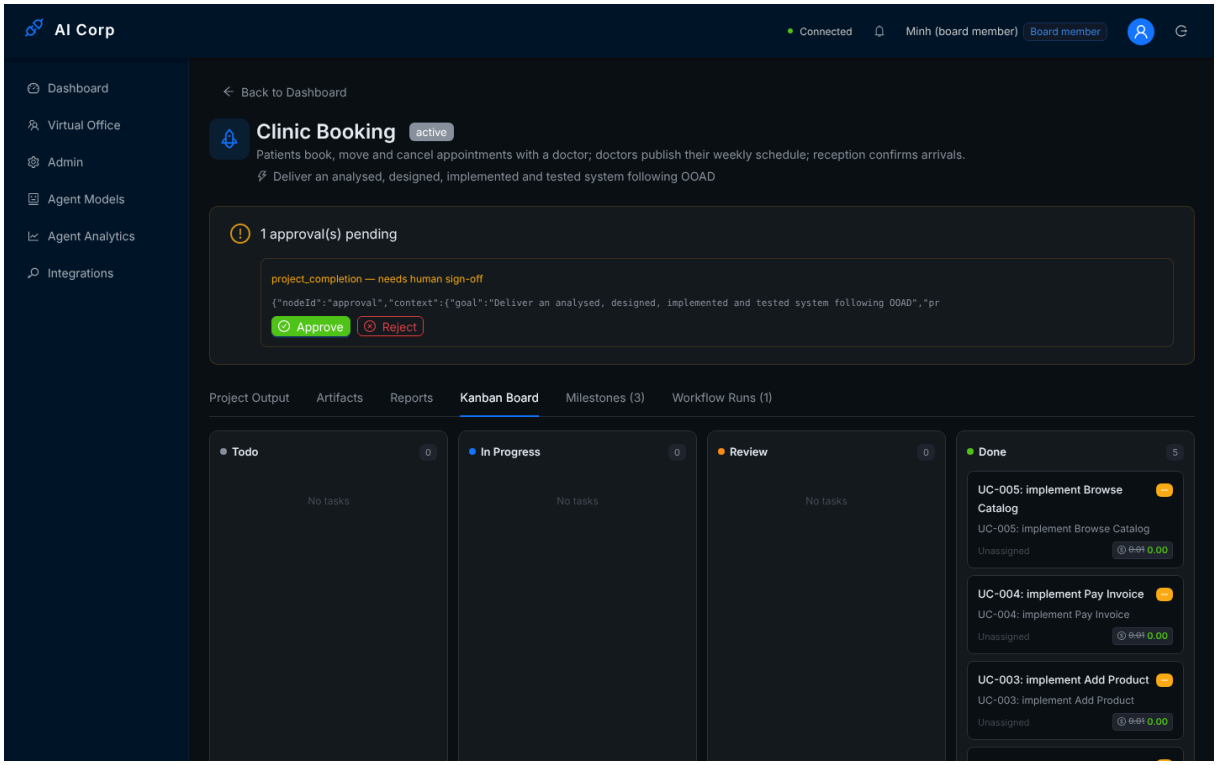
Lần chạy	Nội dung	Kết quả
Triển khai và trình diễn	Cấu hình production qua nginx, dữ liệu trình diễn, phân quyền, phê duyệt, xuất PDF, phân tích, kiểm toán	**21/21**
Phân tích, dự báo, chất lượng mã	8 dự án qua toàn bộ quy trình OOAD rồi 3 lượt chạy khi mô hình của Marketing ngừng hoạt động; bộ nhận webhook thật	**21/21**
Bảo mật, kiểm toán, tích hợp	Vai trò và quyền, nhật ký kiểm toán, khoá API, giới hạn tần suất, webhook ký HMAC, xuất dữ liệu, báo cáo	**44/44**

Ngoài ra có các lần chạy cho bộ máy quy trình (quy trình con, ghim phiên bản, ảnh chụp và khôi phục, nhánh song song) và cho hệ thống tác tử (sản phẩm OOAD được lưu, kiểm thử của QC được thực thi, học, lập trình cặp).

Lần chạy triển khai kiểm tra những điều mà kiểm thử đơn vị không thấy được:

- `prisma migrate deploy` dựng đúng lược đồ trên cơ sở dữ liệu trống, và lược đồ sau đó **không lệch** so với mô hình (`prisma migrate diff --exit-code` trả 0).
- API ở chế độ production từ chối khởi động với cấu hình sai: thiếu `JWT_SECRET` và `CORS_ORIGIN`, khoá LLM là `CHANGE_ME` → thoát với mã 1 và thông báo nêu đủ ba lỗi.
- Qua nginx: kiểm tra sức khoẻ báo cơ sở dữ liệu, Redis và cổng LLM đều *healthy*; khi dừng PostgreSQL, `/api/health` trả **503** còn `/api/health/live` vẫn 200; khởi động lại thì trở về 200.

- Swagger tắt khi `ENABLE_SWAGGER=false`; CORS chỉ trả header cho nguồn gốc được cấu hình.
- WebSocket qua nginx chấp nhận token đăng nhập và ngắt kết nối token giả.
- Người xem không tạo được dự án, không duyệt được qua REST **và không duyệt được qua WebSocket** (lỗ hổng được phát hiện và sửa trong lần chạy này, mục 6.9).
- Sau khi thành viên hội đồng duyệt, CEO đóng dự án; nhật ký kiểm toán có `auth.login`, `project.create`, `approval.requested`, `approval.approve`, `access.denied`, `quality.gate.evaluated`, `workflow.completed`.



Hình 6.1. *Phê duyệt đang chờ trên dự án Clinic Booking*

6.4 Thời gian xử lý của một nút quy trình

Yêu cầu phi chức năng đặt ra là mỗi nút quy trình xử lý trong 2 giây, không tính độ trễ của LLM (độ trễ này phụ thuộc nhà cung cấp và nằm ngoài tầm kiểm soát của nền tảng). Thời lượng của 43 bước tác tử trong lần chạy triển khai (LLM kịch bản trả lời sau 20 ms cộng 1 ms cho mỗi 25 ký tự lời nhắc, nên con số dưới đây vẫn bao gồm một phần độ trễ giả lập):

Vai trò	Số bước	p50 (ms)	p95 (ms)	Lớn nhất (ms)
CEO	10	77	90	92

Vai trò	Số bước	p50 (ms)	p95 (ms)	Lớn nhất (ms)
BA	5	100	109	109
Architect	5	562	580	582
PM	4	96	105	106
DEV	5	603	630	631
QC	5	1 688	1 744	1 756
QA	5	253	269	272
Marketing	4	71	79	80
Tất cả	**43**	**108**	**1 687**	p99: **1 731**

Mọi nút đều dưới 2 giây. Nút chậm nhất là QC vì nó thực sự chạy bộ kiểm thử Jest của dự án sinh ra; Architect và DEV chậm hơn các vai trò khác vì ghi nhiều tệp và chạy phân tích tĩnh.

6.5 Hiệu năng dưới tải

Kiểm thử tải dùng một script không phụ thuộc thư viện, mô phỏng người dùng bảng điều khiển (xem danh sách và chi tiết dự án, công việc, báo cáo chi phí, phân tích tác tử, dự báo, chất lượng mã), có ngân sách p50/p95/p99 cho từng endpoint (100/500/1000 ms mặc định, 300/1500/3000 ms cho phân tích, dự báo và chất lượng mã) và tỉ lệ lỗi tối đa 1%. Môi trường: một tiến trình API, PostgreSQL và Redis trên một máy 4 lõi dùng chung với bộ sinh tải và LLM kịch bản.

Kịch bản	Yêu cầu/giây	Lỗi	Kết quả theo ngân sách
1 người dùng	80,6	0%	Mọi endpoint trong ngân sách; p50 từ 5 đến 39 ms
20 người dùng, chỉ đọc	**201,8**	0%	Mọi endpoint trong ngân sách trừ báo cáo chi phí (p50 102 ms, vượt 2 ms); p95 cao nhất 404 ms
20 người dùng, đồng thời tạo 40 dự án	153,2	0%	p95, p99 đều trong ngân sách; tạo dự án p50 330 ms, p99 622 ms
50 người dùng, tạo dự án	178,3	0%	Bão hoà: p50 ~ 230–360 ms, mọi p99 dưới 1,6 s

Ở 50 người dùng, một tiến trình đã bão hoà ở khoảng 178 yêu cầu/giây, nên theo định luật Little [41] mỗi yêu cầu chờ khoảng 50 / 178 ≈ 280 ms; đây là giới hạn của một tiến trình chứ không phải của truy vấn. API không giữ trạng thái (lượt chạy được điều phối qua BullMQ và PostgreSQL), nên mở rộng là chạy thêm tiến trình.

Kiểm thử tải đã tìm ra ba lỗi hiệu năng (mục 6.9); sau khi sửa, thông lượng ở kịch bản bão hoà tăng từ 98,8 lên 178 yêu cầu/giây, và endpoint chất lượng mã giảm p50 từ 3,1 s xuống 196 ms.

6.6 Độ chính xác của dự báo

Sau 8 dự án hoàn tất, mọi vai trò đủ mẫu để dùng hồi quy. Sai số leave-one-out của thời gian tác tử:

Vai trò	MAPE (LOO)
Architect	3%
CEO	5%
BA	11%
DEV	30%

Dự báo lúc bắt đầu của 7 dự án (dự án đầu tiên chưa có lịch sử) có sai số tuyệt đối trung bình 21% về thời gian và 20% về chi phí. Phần lớn sai số đến từ một dự án mà QA từ chối mã và DEV phải làm lại (chi phí vượt dự báo 84%); các dự án còn lại sai lệch 0–10%. Điều này là đúng bản chất: việc làm lại do bị từ chối không thể biết trước lúc bắt đầu, nên nền tảng lưu cả dự báo lúc bắt đầu và lúc PM lập kế hoạch, và luôn hiển thị dự báo cạnh kết quả thực tế.

6.7 Dự đoán chất lượng mã

Trường hợp	Diễn biến
Legacy Billing	Mã lần đầu của DEV nhập tầng hạ tầng từ miền, chứa khoá viết cứng, gọi `eval` và dồn quy tắc vào một phương thức dài. Điểm 77, có vi phạm phân lớp → dự đoán *fail* → DEV sửa một vòng → điểm 100, bản sửa được giữ, QA duyệt
Unsafe Portal	Vòng sửa không cải thiện điểm nên bản gốc được giữ; QA thấy `eval` và báo cáo phân tích, trả lời FAIL; quy trình quay lại DEV, lần hai đạt
Toàn bộ	9 lần rà soát QA: 8 lần dự đoán *pass* và được duyệt, 1 lần dự đoán *fail* và bị từ chối

Độ chính xác 100% trên 9 lần rà soát cần được hiểu đúng: với LLM kịch bản, các lỗi được cài sẵn đúng loại mà bộ phân tích phát hiện được. Con số này chứng minh chuỗi

dự đoán → sửa → đối chiếu hoạt động, không phải chứng minh độ chính xác với LLM thật (mục 7.4).

6.8 Nghiên cứu tình huống

Năm dự án trình diễn được dựng qua API công khai như người dùng thật, mỗi dự án minh hoạ một khía cạnh:

Dự án	Mẫu quy trình	Kết quả
Online Bookstore	Dự án phần mềm OOAD	5 use case, 3 thực thể, 3 mẫu thiết kế, cổng OOAD 100%; 35 tệp; 4 kiểm thử chấp nhận chạy đạt; độ bao phủ use case 80%, UC-005 được báo thiếu kiểm thử; marketing; hoàn tất
Legacy Billing	Dự án phần mềm OOAD	Phân tích tĩnh phát hiện vi phạm, DEV tự sửa (77 → 100) trước QA; hoàn tất
Unsafe Portal	Dự án phần mềm OOAD	QA từ chối, quy trình quay lại DEV (2 bước DEV, 2 lần rà soát QA); hoàn tất
Library Loans	Chỉ phân tích và thiết kế	Use case, mô hình miền, sơ đồ qua cổng OOAD; hoàn tất
Clinic Booking	Dự án phần mềm OOAD, có phê duyệt	Dừng ở bước phê duyệt; người xem không duyệt được; thành viên hội đồng duyệt, CEO đóng dự án

Cả năm dự án được tạo và chạy xong trong khoảng 25 giây với LLM kịch bản.

6.9 Lỗi phát hiện nhờ kiểm thử

Một kết quả quan trọng của việc kiểm thử trên hạ tầng thật là các lỗi được tìm ra và sửa. Bảng sau tóm tắt theo nhóm (danh sách đầy đủ kèm cách sửa nằm trong kế hoạch dự án của kho mã).

Nhóm	Ví dụ tiêu biểu	Số lỗi
Không khởi động / không chạy được	API không khởi động vì hai chiến lược có tham số hàm tạo mà DI không giải được; migration thiếu 8 bảng; nginx bỏ tiền tố `/api` và không chuyển tiếp WebSocket; Dockerfile chạy tệp không tồn tại	5

Nhóm	Ví dụ tiêu biểu	Số lỗi
Bảo mật	Duyệt qua WebSocket bỏ qua quyền, người được giao và kiểm toán; ba giá trị dự phòng khác nhau cho khoá JWT; cấu hình production thiếu bí mật vẫn khởi động	3
Thiết kế OOAD không được dùng	4/8 tác tử vẫn chạy luồng cũ nên chiến lược là mã chết; cổng chất lượng không được đăng ký; sản phẩm của tác tử mất sau mỗi lệnh sandbox; quy trình lập kế hoạch trước khi có yêu cầu	6
Tính đúng của quy trình và trạng thái	Một máy trạng thái cho mỗi vai trò; nút thất bại để lượt chạy `running` mãi; chuyển trạng thái Kanban không hợp lệ; đóng dự án tạo mốc trùng; `getCostVariance` ném ngoại lệ	8
Chi phí và sự kiện	Chi phí LLM của 4 vai trò không được ghi; vượt ngân sách không tạm dừng quy trình; không có sự kiện `approval.requested` và `workflow.failed`	4
Hiệu năng	Phân tích lại mọi tệp ở mỗi yêu cầu (3,1 s); danh sách dự án tải JSON không dùng; bốn truy vấn tuần tự	3
Tài liệu và giao diện	12/39 sơ đồ UML không render được do lỗi cú pháp; thẻ dự án luôn “0%”; số chi phí chưa làm tròn	3

Lỗ hổng phê duyệt qua WebSocket đáng chú ý vì mọi kiểm thử trước đó đều đi qua REST. Nó chỉ lộ ra khi kiểm thử đi theo đúng đường mà giao diện dùng: người xem nhận 403 từ `POST /api/approvals/:id/approve`, nhưng gửi thông điệp `human:approval_response` qua WebSocket thì phê duyệt được chấp nhận, dự án được đóng, và không có mục kiểm toán nào. Bài học thiết kế là mọi đường ghi phải đi qua cùng một điểm kiểm soát; đường WebSocket nay từ chối thông điệp này, giao diện dùng REST, và lần chạy đầu-cuối có kiểm tra riêng cho trường hợp này.

Chương 7
THẢO LUẬN

Chương này trả lời ba câu hỏi nghiên cứu nêu ở chương 1 dựa trên kết quả của chương 6, sau đó phân tích các mối đe doạ đến tính hợp lệ của đánh giá và các giới hạn của nền tảng.

7.1 RQ1: OOAD có cải thiện kiến trúc của hệ đa tác tử dùng LLM không?

Câu trả lời là có, với một điều kiện quan trọng được nêu ở cuối mục này. Bằng chứng được xét theo ba thuộc tính.

Tính mô-đun. Tầng miền gồm 101 tệp (6 893 dòng) không nhập NestJS, Prisma hay thư viện LLM nào; mọi phụ thuộc hạ tầng đi qua giao diện Repository hoặc cổng (port) như `ICodeAnalyzer`. Các mô-đun giao tiếp bằng sự kiện miền qua `EventBus`, nên kiểm toán, webhook và phân tích hiệu năng được thêm vào mà không sửa nơi phát sự kiện. Trong quá trình phát triển, nhiều thay đổi lớn chỉ chạm một chỗ: ghi chi phí cho cả tám vai trò được giải quyết bằng một Decorator quanh nhà cung cấp LLM; đóng lỗ hổng phê duyệt qua WebSocket là xoá một đường ghi và chuyển giao diện sang REST, phần kiểm quyền và kiểm toán không đổi.

Khả năng mở rộng. Thêm một vai trò tác tử gồm một giá trị `AgentRole`, một `RoleProfile`, hai lớp chiến lược và một dòng đăng ký nhà cung cấp; `AgentOrchestrator` và `WorkflowEngine` không đổi (mục 5.3). Thêm một loại nút quy trình là thêm một `NodeExecutionStrategy` tự đăng ký; quy trình con, cổng chất lượng, phê duyệt và ba mẫu cộng tác đều được thêm theo cách này. Thêm một định dạng xuất, một quy tắc bất thường hay một tín hiệu chất lượng cũng là thêm một lớp vào sổ đăng ký.

Khả năng kiểm thử. Vì miền độc lập framework, các quy tắc nghiệp vụ được kiểm thử bằng hàm thuần, không giả lập, và tầng miền đạt 97,1% dòng và 89,6% nhánh (mục 6.2). Chiến lược tác tử được kiểm thử với LLM và sandbox giả lập qua giao diện; bộ máy quy trình chạy được cả có và không có Redis. Toàn API có độ bao phủ thấp hơn (77,0% dòng) do phần nối framework, và đây là phân bố hợp lý: phần chứa quyết định được kiểm thử kỹ nhất.

Điều kiện. Cấu trúc OOAD tốt trong mã không bảo đảm thiết kế được dùng. Mục 6.9 cho thấy sáu lỗi thuộc nhóm “thiết kế OOAD không được dùng”: bốn trên tám tác tử

vẫn chạy luồng cũ nên các lớp chiến lược của chúng là mã chết, cổng chất lượng được viết nhưng không được đăng ký vào bộ máy, sản phẩm của tác tử bị mất sau mỗi lệnh sandbox. Mọi kiểm thử đơn vị của các lớp này đều đạt. Chỉ kiểm thử đầu-cuối trên hạ tầng thật, kiểm tra kết quả nhìn thấy được (tệp trên đĩa, mục kiểm toán, trạng thái lượt chạy) mới phát hiện ra. Kết luận rút ra là OOAD cải thiện kiến trúc khi đi kèm kiểm thử chứng minh từng đường thiết kế thực sự được thực thi, chẳng hạn kiểm tra rằng mỗi vai trò đều đi qua `BaseAgent.execute()` và mỗi chiến lược đăng ký đều được gọi.

7.2 RQ2: Những mẫu thiết kế nào hiệu quả nhất cho việc điều phối?

Mức độ hiệu quả ở đây được đánh giá bằng tiêu chí thực dụng: mẫu nào giữ cho thay đổi cục bộ và mẫu nào giúp tìm ra hoặc ngăn lỗi trong quá trình phát triển.

1. **Template Method kết hợp Strategy cho tác tử** là mẫu có giá trị nhất. Chu trình chung (sự kiện, chuyển trạng thái, bộ nhớ, xử lý lỗi) nằm ở một nơi, còn chuyên môn của từng vai trò nằm trong hai chiến lược thay được độc lập. Khi phát hiện bốn tác tử chạy luồng cũ, việc sửa là đưa chúng về cùng khung, và mọi tính năng gắn với khung (ghi chi phí, học, sự kiện) tự động có hiệu lực cho cả bốn.
2. **Strategy tự đăng ký cho nút quy trình**, cùng **Composite** cho quy trình con, cho phép bộ máy quy trình phát triển từ chuỗi tuyến tính thành đồ thị có nhánh song song, quy trình con, phê duyệt và cộng tác mà không sửa vòng lặp thực thi.
3. **State**, với một bài học về phạm vi: máy trạng thái phải gắn với đối tượng thực sự có vòng đời. Gắn một máy trạng thái cho mỗi vai trò làm hai công việc đồng thời phá nhau; gắn cho mỗi lần thực thi thì đúng (mục 4.3.2). Với công việc và dự án, việc đặt bất biến chuyển trạng thái trong thực thể đã chặn các chuyển trạng thái Kanban không hợp lệ ngay tại miền.
4. **Observer / sự kiện miền** cho các mối quan tâm cắt ngang: kiểm toán, webhook, phân tích hiệu năng và thông báo thời gian thực đều đăng ký nghe sự kiện thay vì được gọi trực tiếp. Hai lỗi “thiếu sự kiện” (mục 6.9) cũng cho thấy mặt trái: một sự kiện không được phát thì mọi bên nghe đều im lặng, và chỉ kiểm thử đầu-cuối thấy được điều đó.
5. **Memento và sổ đăng ký bất biến** cho phiên bản quy trình làm cho việc khôi phục và ghim phiên bản trở nên đơn giản, vì lượt chạy không bao giờ thấy định nghĩa thay đổi dưới chân mình.
6. **Mediator** (`AgentMessageBus`) giữ cho các tác tử không biết nhau khi cộng tác, và tạo một điểm duy nhất để giới hạn số vòng và hiển thị hội thoại.

Các mẫu ít ảnh hưởng hơn đến điều phối nhưng hữu ích cho phần còn lại là Decorator (chi phí), Facade (một điểm gọi LLM), Chain of Responsibility (guard và chuỗi rà soát) và Port/Adapter (phân tích mã). Một quan sát chung là mẫu thiết kế hiệu quả nhất khi nó tạo ra **một điểm kiểm soát duy nhất**. Lỗ hổng phê duyệt qua WebSocket xuất hiện chính vì có một đường ghi thứ hai đi vòng qua điểm kiểm soát đó.

7.3 RQ3: Tác tử học từ kết quả như thế nào, và đo điều đó ra sao?

Nền tảng có hai dạng học và chúng được đo theo cách khác nhau.

Học để dự báo dùng lịch sử các lượt chạy để ước lượng thời gian và chi phí. Dạng này đo được trực tiếp bằng sai số leave-one-out: sau tám dự án, MAPE theo vai trò từ 3% (Architect) đến 30% (DEV), và dự báo lúc bắt đầu sai trung bình khoảng 20% (mục 6.6). Sai số lớn nhất đến từ việc làm lại sau khi QA từ chối, một sự kiện không thể biết trước lúc bắt đầu. Vì vậy nền tảng hiển thị dự báo cạnh thực tế thay vì coi dự báo là cam kết.

Học để cải thiện sản phẩm là việc tinh luyện kết quả khách quan (cổng OOAD, kiểm thử của QC, phán quyết QA) thành bài học và nạp bài học vào lời nhắc của lần sau (mục 4.3.4). Các lần chạy đầu-cuối kiểm chứng rằng chuỗi này hoạt động đúng: kết quả được ghi cho đúng vai trò, bài học được tinh luyện sau mỗi N kết quả và xuất hiện trong lời nhắc tiếp theo. Tuy nhiên **luận văn không đo được mức cải thiện chất lượng nhờ học**. Với LLM kịch bản, câu trả lời không phụ thuộc vào bài học trong lời nhắc, nên mọi so sánh trước và sau sẽ cho kết quả bằng nhau theo cấu tạo. Báo cáo một con số cải thiện trong điều kiện đó sẽ là không trung thực.

Cách đo đúng được đề xuất như sau và là hướng phát triển đầu tiên ở chương 8: dùng LLM thật với nhiệt độ cố định; một tập dự án cố định chia hai nhóm, có học và không học (`LEARNING_REFINE_EVERY` tắt); chạy mỗi nhóm nhiều lần; so sánh tỉ lệ đạt cổng OOAD ngay lần đầu, số lần QA từ chối, số vòng sửa và chi phí làm lại, kèm kiểm định thống kê. Hạ tầng cần cho thí nghiệm này đã có sẵn: tín hiệu kết quả, phân tích hiệu năng theo vai trò và ghi chi phí từng lời gọi.

7.4 Mối đe doạ đến tính hợp lệ của đánh giá

Phần này theo cách phân loại thường dùng trong nghiên cứu thực nghiệm kỹ nghệ phần mềm: hợp lệ nội tại, hợp lệ cấu trúc và hợp lệ ngoại tại.

Hợp lệ nội tại: LLM kịch bản. Mọi lần chạy đầu-cuối dùng một máy chủ LLM kịch bản tương thích OpenAI. Nó trả lời theo nội dung lời nhắc (tên dự án, vai trò, bước),

có độ trễ và giá giả lập. Lựa chọn này làm cho kết quả lặp lại được và cho phép kiểm thử chính xác các nhánh khó tạo ra với LLM thật, như mã vi phạm phân lớp, QA từ chối hay mô hình ngừng hoạt động. Đổi lại, nó giới hạn những gì kết quả chứng minh được:

- Các kết quả về **luồng điều khiển** là hợp lệ: quy trình đi đúng nhánh, trạng thái được lưu, quyền được kiểm, chi phí được ghi, lỗi được xử lý. Chúng không phụ thuộc nội dung câu trả lời.
- Các kết quả về **chất lượng nội dung** chỉ là minh hoạ: độ chính xác 100% của dự đoán chất lượng mã (mục 6.7) đạt được vì lỗi được cài sẵn đúng loại bộ phân tích phát hiện được; số use case, số tệp và độ bao phủ use case của các dự án trình diễn phản ánh kịch bản chứ không phản ánh năng lực của một mô hình thật.
- Các con số **thời gian** bao gồm độ trễ giả lập (20 ms cộng 1 ms cho mỗi 25 ký tự lời nhắc). Với LLM thật, độ trễ mỗi lời gọi thường tính bằng giây và chi phối tổng thời gian; con số 2 giây cho mỗi nút được định nghĩa là không tính độ trễ LLM vì lý do này.
- Mô hình dự báo được huấn luyện trên thời lượng và chi phí do kịch bản sinh ra, nên MAPE thấp của Architect và CEO một phần là do hành vi giả lập ổn định. Với LLM thật, phương sai sẽ lớn hơn và sai số dự kiến cao hơn.

Hợp lệ cấu trúc. “Chất lượng OOAD” của dự án sinh ra được đo bằng cổng chất lượng và bộ phân tích tĩnh do chính luận văn định nghĩa: có đủ use case, thực thể, sơ đồ, không vi phạm phân lớp, độ phức tạp trong ngưỡng. Đây là các thước đo cần, chưa phải đủ; một thiết kế qua được cổng vẫn có thể là thiết kế kém theo đánh giá của chuyên gia. Tương tự, độ bao phủ mã đo dòng được thực thi chứ không đo chất lượng của khẳng định trong kiểm thử.

Hợp lệ ngoại tại. Kiểm thử tải chạy một tiến trình API trên một máy 4 lõi dùng chung với bộ sinh tải; con số thông lượng là giới hạn dưới cho cấu hình đó, không phải dự đoán cho một cụm máy. Các dự án sinh ra đều là ứng dụng TypeScript cỡ nhỏ (khoảng 35 tệp); kết quả chưa cho thấy nền tảng xử lý được dự án lớn hay ngôn ngữ khác. Chưa có nghiên cứu với người dùng thật nên không có kết luận về mức hữu ích đối với một nhóm phát triển.

Giảm thiểu. Để hạn chế các mối đe doạ trên, luận văn chỉ báo cáo số đo từ các lần chạy có lưu vết, tách rõ kết quả về luồng điều khiển với kết quả về nội dung, và công

bố mã của LLM kịch bản cùng các script đo để người khác tái tạo hoặc thay bằng LLM thật (Phụ lục C và D).

7.5 Các giới hạn của nền tảng

- **Sandbox:** môi trường kiểm thử không có Docker daemon, nên 15 kiểm thử của sandbox container bị bỏ qua và các lần chạy đầu-cuối chạy kiểm thử của dự án sinh ra trong một bản sao tạm trên máy chủ (`SANDBOX_ALLOW_LOCAL_RUN=true`). Cách này an toàn ở đây vì mã do LLM kịch bản viết, nhưng với LLM thật phải dùng sandbox container, và đường chạy đó chưa được kiểm chứng đầu-cuối.
- **Ngôn ngữ đích:** mẫu sản phẩm, bộ phân tích tĩnh và việc chạy kiểm thử chấp nhận chỉ hỗ trợ TypeScript.
- **Con người trong vòng lặp:** phê duyệt là một nút chặn đơn giản (duyệt hoặc từ chối kèm ghi chú). Chưa có chỉnh sửa sản phẩm trực tiếp rồi cho quy trình tiếp tục từ đó.
- **Mở rộng ngang:** API không giữ trạng thái nên có thể chạy nhiều tiến trình, nhưng cấu hình triển khai hiện chạy một tiến trình và chưa được kiểm thử tải ở chế độ nhiều bản sao. Kết nối WebSocket nhiều bản sao cần thêm bộ chuyển Redis cho Socket.IO.
- **Bảo mật:** các kiểm soát (phân quyền, khoá API, giới hạn tần suất, ký HMAC, kiểm toán, kiểm tra cấu hình production) đã được kiểm thử chức năng nhưng chưa qua kiểm thử xâm nhập độc lập. Lỗ hổng WebSocket ở mục 6.9 cho thấy giá trị của việc rà soát mọi đường ghi.
- **Chi phí:** ngân sách dừng quy trình khi vượt, nhưng chưa có ước lượng chi phí trước mỗi lời gọi để chặn sớm.

Chương 8
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

8.1 Kết luận

Luận văn đã xây dựng AI Corp, một nền tảng đa tác tử dùng LLM mô phỏng một công ty phần mềm với tám vai trò (CEO, BA, Architect, PM, DEV, QC, QA, Marketing), và áp dụng phân tích và thiết kế hướng đối tượng ở ba cấp: kiến trúc của nền tảng, thiết kế của từng tác tử, và sản phẩm mà các tác tử tạo ra cho người dùng.

Về kiến trúc, nền tảng có tầng miền độc lập framework, tác tử là Template Method cùng hai Strategy, máy trạng thái cho tác tử, công việc và dự án, sự kiện miền cho giao tiếp giữa các mô-đun, và Repository cùng cổng cho mọi phụ thuộc hạ tầng. Bộ máy quy trình hỗ trợ quy trình con, phiên bản bất biến, ảnh chụp và khôi phục, nhánh song song, phê duyệt của con người và ba mẫu cộng tác giữa tác tử. Về sản phẩm, mỗi dự án đi qua chuỗi use case → mô hình miền → UML → mã phân lớp → kiểm thử chấp nhận → ma trận truy vết, với cổng chất lượng OOAD, phân tích tĩnh và một vòng tự sửa có giới hạn chi phí. Về vận hành, nền tảng có phân quyền theo vai trò, nhật ký kiểm toán, khoá API, webhook ký HMAC, xuất báo cáo, phân tích hiệu năng tác tử, dự báo thời gian và chi phí, cùng cấu hình triển khai Docker có kiểm tra cấu hình production và kiểm tra sức khoẻ.

Các kết quả chính đo được:

- 674 kiểm thử API và 41 kiểm thử web đạt; tầng miền có độ bao phủ 97,1% dòng và 89,6% nhánh.
- Các lần chạy đầu-cuối trên hạ tầng thật đạt 21/21, 21/21 và 44/44 kiểm tra; cấu hình production dựng được lược đồ từ migration không lệch so với mô hình.
- Mọi nút quy trình xử lý dưới 2 giây (p95 1,69 s, chưa trừ độ trễ LLM giả lập); một tiến trình API phục vụ khoảng 200 yêu cầu/giây chỉ đọc và 178 yêu cầu/giây khi bão hoà, không có lỗi.
- Dự báo thời gian tác tử có MAPE leave-one-out từ 3% đến 30% tuỳ vai trò.
- Kiểm thử trên hạ tầng thật phát hiện và sửa 32 lỗi, trong đó có một lỗ hổng cho phép người xem phê duyệt qua WebSocket.

Với ba câu hỏi nghiên cứu, luận văn kết luận: (1) OOAD cải thiện rõ tính mô-đun, khả năng mở rộng và khả năng kiểm thử, với điều kiện có kiểm thử đầu-cuối chứng minh

thiết kế thực sự được dùng; (2) Template Method kết hợp Strategy cho tác tử, Strategy tự đăng ký kết hợp Composite cho quy trình, và State gắn đúng đối tượng có vòng đời là các mẫu hiệu quả nhất, và nhìn chung mẫu có giá trị nhất khi tạo ra một điểm kiểm soát duy nhất; (3) học từ lịch sử để dự báo đo được và cho sai số chấp nhận được, còn học để cải thiện sản phẩm đã được hiện thực và kiểm chứng về cơ chế, nhưng mức cải thiện chỉ có thể đo trung thực với LLM thật trong một thí nghiệm có nhóm đối chứng.

8.2 Hướng phát triển

1. **Đánh giá với LLM thật.** Chạy lại tập dự án đánh giá với một hoặc nhiều mô hình thật qua cổng tương thích OpenAI, đo chất lượng sản phẩm (tỉ lệ đạt cổng OOAD lần đầu, số lần QA từ chối, kiểm thử chấp nhận đạt), thời gian và chi phí thực, và đánh giá lại mô hình dự báo và bộ dự đoán chất lượng mã trên dữ liệu đó.
2. **Thí nghiệm có đối chứng cho cơ chế học**, theo thiết kế ở mục 7.3: hai nhóm có học và không học, nhiều lần lặp, kiểm định thống kê.
3. **Đánh giá thiết kế bởi chuyên gia**: cho chuyên gia chấm sản phẩm OOAD của các dự án sinh ra theo một thang đo, để kiểm tra xem cổng chất lượng tự động có tương quan với đánh giá của con người không.
4. **Sandbox container trong kiểm thử thường xuyên**: chạy các kiểm thử sandbox trên một máy chạy CI có Docker, và kiểm chứng đầu-cuối đường chạy mã trong container.
5. **Mở rộng ngang**: thêm bộ chuyển Redis cho Socket.IO, chạy nhiều bản sao API sau nginx, và kiểm thử tải ở cấu hình đó.
6. **Ngôn ngữ đích khác**: tách bộ phân tích tĩnh và trình chạy kiểm thử thành các adapter theo ngôn ngữ sau cổng `ICodeAnalyzer` đã có, bắt đầu với Python hoặc Java.
7. **Con người trong vòng lặp sâu hơn**: cho phép người dùng sửa trực tiếp một sản phẩm (ví dụ mô hình miền) tại bước phê duyệt và để quy trình tiếp tục từ bản đã sửa, tận dụng cơ chế ảnh chụp và khôi phục.
8. **Kiểm thử xâm nhập độc lập** cho API, WebSocket và webhook trước khi dùng với dữ liệu thật.

TÀI LIỆU THAM KHẢO

[1] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2005.

[2] Object Management Group, *OMG Unified Modeling Language (OMG UML), Version 2.5.1*, formal/17-12-05, Dec. 2017.

[3] I. Jacobson, M. Christerson, P. Jonsson, and G. Övergaard, *Object-Oriented Software Engineering: A Use Case Driven Approach*. Wokingham, U.K.: Addison-Wesley, 1992.

[4] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[5] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[6] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.

[7] R. C. Martin, "Design principles and design patterns," Object Mentor, 2000.

[8] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.

[9] C. Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2004.

[10] B. Meyer, *Object-Oriented Software Construction*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1997.

[11] B. H. Liskov and J. M. Wing, "A behavioral notion of subtyping," *ACM Trans. Program. Lang. Syst.*, vol. 16, no. 6, pp. 1811–1841, 1994.

[12] P. B. Kruchten, "The 4+1 view model of architecture," *IEEE Software*, vol. 12, no. 6, pp. 42–50, 1995.

[13] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2012.

[14] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, U.K.: Wiley, 2009.

[15] A. S. Rao and M. P. Georgeff, "BDI agents: From theory to practice," in *Proc. 1st Int. Conf. Multi-Agent Systems (ICMAS-95)*, San Francisco, CA, USA, 1995, pp. 312–319.

[16] Foundation for Intelligent Physical Agents, “FIPA ACL message structure specification,” SC00061G, 2002.

[17] R. G. Smith, “The contract net protocol: High-level communication and control in a distributed problem solver,” *IEEE Trans. Comput.*, vol. C-29, no. 12, pp. 1104–1113, 1980.

[18] P. Stone and M. Veloso, “Multiagent systems: A survey from a machine learning perspective,” *Autonomous Robots*, vol. 8, no. 3, pp. 345–383, 2000.

[19] Object Management Group, *Business Process Model and Notation (BPMN), Version 2.0*, formal/2011-01-03, Jan. 2011.

[20] T. Murata, “Petri nets: Properties, analysis and applications,” *Proc. IEEE*, vol. 77, no. 4, pp. 541–580, 1989.

[21] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, “Workflow patterns,” *Distributed and Parallel Databases*, vol. 14, no. 1, pp. 5–51, 2003.

[22] M. Chen *et al.*, “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.

[23] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[24] S. Hong *et al.*, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[25] C. Qian *et al.*, “ChatDev: Communicative agents for software development,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.

[26] Q. Wu *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” arXiv:2308.08155, 2023.

[27] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[28] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[29] J. S. Park *et al.*, “Generative agents: Interactive simulacra of human behavior,” in *Proc. 36th Annu. ACM Symp. User Interface Software and Technology (UIST)*, 2023.

[30] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[31] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[32] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[33] D. Harel, “Statecharts: A visual formalism for complex systems,” *Sci. Comput. Program.*, vol. 8, no. 3, pp. 231–274, 1987.

[34] K. Beck, *Test-Driven Development: By Example*. Boston, MA, USA: Addison-Wesley, 2002.

[35] T. J. McCabe, “A complexity measure,” *IEEE Trans. Softw. Eng.*, vol. SE-2, no. 4, pp. 308–320, 1976.

[36] M. H. Halstead, *Elements of Software Science*. New York, NY, USA: Elsevier North-Holland, 1977.

[37] P. Oman and J. Hagemeister, “Metrics for assessing a software system’s maintainability,” in *Proc. Conf. Software Maintenance*, Orlando, FL, USA, 1992, pp. 337–344.

[38] D. Coleman, D. Ash, B. Lowther, and P. Oman, “Using metrics to evaluate software system maintainability,” *Computer*, vol. 27, no. 8, pp. 44–49, 1994.

[39] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ, USA: Prentice Hall, 1981.

[40] A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[41] J. D. C. Little, “A proof for the queuing formula: L = λW,” *Oper. Res.*, vol. 9, no. 3, pp. 383–387, 1961.

[42] *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.

[43] *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*, ISO/IEC 25010:2011, 2011.

[44] O. C. Z. Gotel and A. C. W. Finkelstein, “An analysis of the requirements traceability problem,” in *Proc. 1st Int. Conf. Requirements Engineering*, Colorado Springs, CO, USA, 1994, pp. 94–101.

[45] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[46] R. Kohavi, “A study of cross-validation and bootstrap for accuracy estimation and model selection,” in *Proc. 14th Int. Joint Conf. Artificial Intelligence (IJCAI)*, 1995, pp. 1137–1145.

[47] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[48] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-hashing for message authentication,” RFC 2104, Feb. 1997.

[49] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, May 2015.

[50] OWASP Foundation, “OWASP Top 10:2021,” 2021.

[51] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. Hoboken, NJ, USA: Wiley, 2011.

[52] M. Cohn, *Succeeding with Agile: Software Development Using Scrum*. Boston, MA, USA: Addison-Wesley, 2009.

[53] *Quality Management Systems — Fundamentals and Vocabulary*, ISO 9000:2015, 2015.

[54] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.

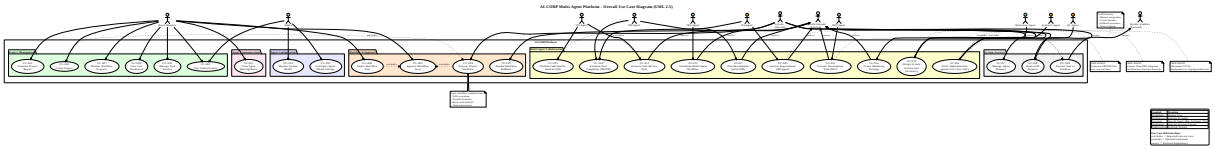
[55] D. L. Parnas, “On the criteria to be used in decomposing systems into modules,” *Commun. ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.

Phụ lục A. Danh mục sơ đồ UML

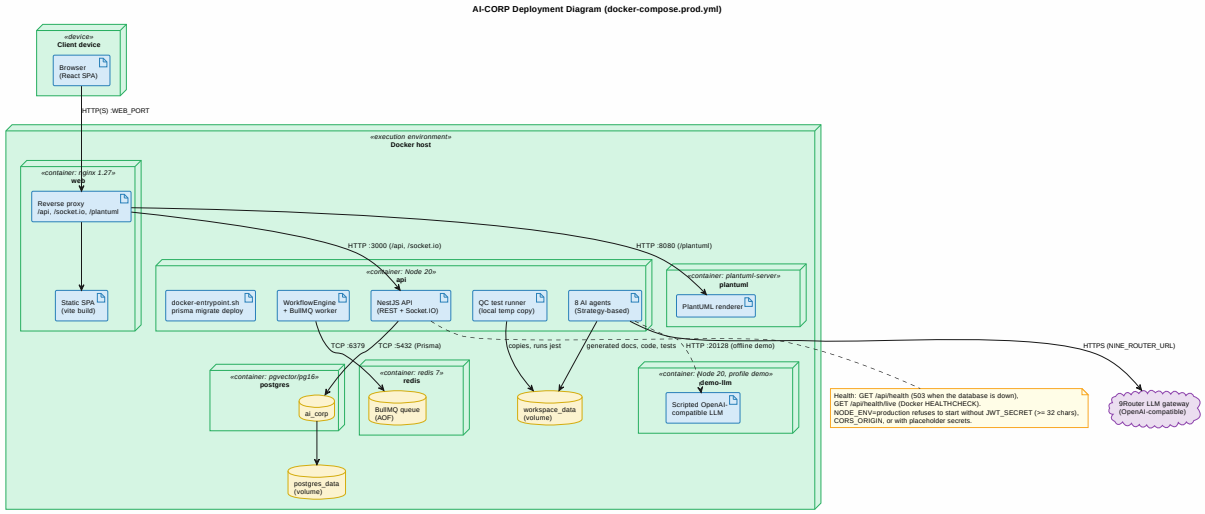
Các sơ đồ là mã nguồn PlantUML trong `docs/diagrams/`. Lệnh `scripts/docs/render-diagrams.sh` tải PlantUML (kiểm tra mã băm SHA-1), render toàn bộ sang SVG trong `docs/diagrams/rendered/` và dừng với mã lỗi khác 0 ở sơ đồ đầu tiên có lỗi cú pháp. Danh mục đầy đủ có liên kết nằm ở `docs/diagrams/INDEX.md`.

Góc nhìn	Số sơ đồ	Sơ đồ
Use case	4	Toàn hệ thống; quản trị viên; thành viên hội đồng; tác tử
Lớp	13	Tổng quan mô hình miền; aggregate Agent, Project, Workflow; Strategy, Observer, Repository, Factory; vòng học; cộng tác tác tử; bộ máy quy trình; bảo mật, kiểm toán, tích hợp, báo cáo; phân tích, dự báo, chất lượng mã
Tuần tự	11	SD-001 tạo dự án; SD-002 chu trình suy nghĩ–hành động; SD-003 phê duyệt; SD-004 chuỗi giao việc; SD-005 học; SD-006 ảnh chụp và khôi phục; SD-007 lập trình cặp; SD-008 quy trình con; SD-009 gọi API bằng khoá API; SD-010 gửi webhook; SD-011 xuất dự án
Hoạt động	4	AD-001 vòng đời dự án; AD-002 quy trình có nhánh song song; AD-003 ước lượng công sức và chi phí; AD-004 phân tích và sửa chất lượng mã

Góc nhìn	Số sơ đồ	Sơ đồ
Máy trạng thái	4	SM-001 tác tử; SM-002 lượt chạy quy trình; SM-003 công việc; SM-004 phê duyệt
Kiến trúc	3	Gói; thành phần; triển khai
Tổng	**39**	



Hình A.1. *Sơ đồ use case tổng thể*



Hình A.2. *Sơ đồ triển khai*

Phụ lục B. Giao diện lập trình (API)

Mọi đường dẫn có tiền tố `/api`. Trừ `auth/register`, `auth/login` và `health`, mọi route yêu cầu JWT (giao diện web) hoặc khoá API (nhóm `/v1`), và được kiểm quyền theo vai trò. Khi `ENABLE_SWAGGER=true`, tài liệu OpenAPI đầy đủ có tại `/api/docs`.

Nhóm	Route
Xác thực	`POST auth/register`, `POST auth/login`, `GET auth/me`
Dự án	`GET/POST projects`, `GET/PATCH/DELETE projects/:id`
Công việc, mốc	`GET tasks`, `POST projects/:projectId/tasks`, `PATCH tasks/:id`; `GET/POST projects/:projectId/milestones`, `PATCH milestones/:id`
Sản phẩm của dự án	`GET projects/:id/files`, `GET projects/:id/files/*`, `GET projects/:id/archive` (zip), `GET projects/:id/quality-gates`
Phê duyệt	`GET approvals`, `POST approvals/:id/approve`, `POST approvals/:id/reject`
Quy trình	`GET workflows/templates`, `POST workflows/templates/:templateId/preview`, `GET workflows/definitions[/:definitionId]`, `GET workflows/runs/:runId`, `GET workflows/runs/:runId/snapshots`, `POST workflows/runs/:runId/rollback`, `POST workflows/runs/:runId/pause`, `POST workflows/runs/:runId/resume`
Mô hình LLM	`GET models`, `GET models/available`, `PATCH models/:role`
Chi phí, báo cáo, xuất	`GET cost/summary`, `GET reports/projects/:id/cost`, `GET reports/performance`, `POST reports/weekly/run`, `GET projects/:id/export?format=json|csv|pdf`

Nhóm	Route
Phân tích, dự báo, chất lượng	`GET analytics/agents`, `GET agents/:role/learning`, `GET projects/:id/estimate`, `GET analytics/estimation`, `GET projects/:id/code-quality`, `GET ooad-metrics/dashboard`, `GET ooad-metrics/project/:id`, `GET ooad-metrics/leaderboard`
Người dùng, kiểm toán	`GET users`, `GET users/roles`, `PATCH users/:id/role`, `PATCH users/:id/active`; `GET audit`, `GET audit/export`, `GET audit/anomalies`
Tích hợp (quản trị)	`GET integrations/events`, `GET/POST integrations/api-keys`, `DELETE integrations/api-keys/:id`, `GET/POST integrations/webhooks`, `DELETE integrations/webhooks/:id`, `POST integrations/webhooks/:id/test`, `GET integrations/webhooks/:id/deliveries`
API công khai (khoá API)	`GET/POST v1/projects`, `GET v1/projects/:id/status`, `GET v1/projects/:id/reports/cost`, `GET v1/projects/:id/export`, `GET/POST v1/webhooks`, `DELETE v1/webhooks/:id`
Sức khoẻ	`GET health` (503 khi cơ sở dữ liệu ngừng), `GET health/live`

WebSocket (Socket.IO, xác thực bằng JWT khi kết nối): máy khách gửi `subscribe_project` để nhận `project_event` của một dự án; máy chủ phát thêm `system_event` (trạng thái tác tử) và `user_event`. Thông điệp `human:approval_response` bị từ chối; phê duyệt chỉ đi qua REST (mục 6.9).

Phụ lục C. Số liệu và cách tái tạo kiểm thử

Kiểm thử tự động

```
        pnpm install
        cd apps/api && npx jest                    # 47 bộ, 674 đạt, 15 bỏ qua
        cd apps/api && npm run test:cov            # độ bao phủ, thất bại nếu
dưới ngưỡng
        cd apps/web && npx vitest run && npx tsc --noEmit
```

Kiểm thử đầu-cuối

Mỗi lần chạy có script, kết quả và ảnh chụp trong một thư mục `docs/*-e2e/` của kho mã, kèm README ghi môi trường và lệnh chạy. Script in một dòng cho mỗi kiểm tra và trả mã 1 khi có kiểm tra thất bại.

Lần chạy	Script	Kiểm tra
Triển khai và trình diễn	`docs/phase7-e2e/e2e.mjs` sau `scripts/demo/seed-demo.mjs`	21
Phân tích, dự báo, chất lượng mã	`docs/phase6-e2e/e2e.mjs`	21
Bảo mật, kiểm toán, tích hợp	`docs/phase5-e2e/e2e.mjs`	44
Bộ máy quy trình (quy trình con, ghim phiên bản, ảnh chụp, khôi phục)	`docs/phase4-e2e/e2e.mjs`	theo kịch bản

Môi trường chung: PostgreSQL 16 có pgvector, Redis 7, API build ở chế độ production, LLM kịch bản `scripts/demo/demo-llm.cjs` (tương thích OpenAI, cổng 20128). Lần chạy triển khai đi qua nginx với cấu hình giống `apps/web/nginx.conf`.

LLM kịch bản

Máy chủ trả lời theo vai trò và nội dung lời nhắc. Các quy tắc có ảnh hưởng đến kết quả đánh giá:

Điều kiện	Câu trả lời
Tên dự án bắt đầu bằng “Legacy”	Mã lần đầu của DEV nhập hạ tầng từ miền, có khoá viết cứng, `eval` và phương thức dài; lần sửa thì sạch

Điều kiện	Câu trả lời
Tên dự án bắt đầu bằng “Unsafe”	Mã có `eval`; QA trả lời FAIL ở lần rà soát đầu, PASS ở lần hai
Mô hình của Marketing bị đánh dấu ngừng hoạt động	Lỗi 503, dùng để kiểm tra phát hiện bất thường
Mọi lời gọi	Độ trễ 20 ms + 1 ms cho mỗi 25 ký tự lời nhắc; số token theo độ dài; giá theo tên mô hình

Dữ liệu trình diễn

`scripts/demo/seed-demo.mjs` tạo hai tài khoản (thành viên hội đồng, người xem) và năm dự án ở mục 6.8. Script lặp lại được: chạy lần hai không tạo bản trùng.

Phụ lục D. Số liệu hiệu năng

Kiểm thử tải

Script `scripts/perf/load-test.mjs` (không phụ thuộc thư viện), ngân sách trong `scripts/perf/budgets.json`:

```
        node scripts/perf/load-test.mjs --api http://localhost:3000/api --users
20 --duration 30 --create-every 0
        node scripts/perf/load-test.mjs --users 20 --duration 30 --create-every
10   # có quy trình đang chạy
        node scripts/perf/load-test.mjs --users 50 --duration 30 --create-every
25   # bão hoà
```

Kết quả chi tiết với 20 người dùng chỉ đọc (30 s, 6 078 yêu cầu, 201,8 yêu cầu/giây, 0% lỗi):

Endpoint	Số yêu cầu	p50 (ms)	p95 (ms)	p99 (ms)	Ngân sách p50/p95/p99
`GET /analytics/agents`	578	128	199	229	300/1500/3000
`GET /health`	300	81	154	187	100/500/1000
`GET /projects`	1 505	79	148	182	100/500/1000
`GET /projects/:id`	1 242	86	150	187	100/500/1000
`GET /projects/:id/code-quality`	615	58	404	556	300/1500/3000
`GET /projects/:id/estimate`	623	130	219	257	300/1500/3000
`GET /reports/projects/:id/cost`	628	102	177	222	100/500/1000
`GET /tasks?projectId`	587	76	136	178	100/500/1000

Ba lỗi hiệu năng tìm được nhờ kiểm thử tải và hiệu quả của bản sửa (cùng kịch bản 50 người dùng có tạo dự án):

Lỗi	Cách sửa	Kết quả
Endpoint chất lượng mã phân tích lại mọi tệp ở mỗi yêu cầu	Lưu kết quả theo mã băm nội dung tệp và theo dự án trong 30 s; DEV xoá bộ đệm khi ghi lại mã; đọc tệp song song	p50 3 116 → 196 ms
Danh sách dự án tải JSON ngữ cảnh của lượt chạy rồi bỏ đi	Repository chỉ chọn các cột DTO cần	46 → 9 ms (1 người dùng)

Lỗi	Cách sửa	Kết quả
Chi tiết dự án chạy bốn truy vấn độc lập tuần tự	Chạy song song	16 → 9 ms (1 người dùng)

Thông lượng ở kịch bản bão hoà tăng từ 98,8 lên 178,3 yêu cầu/giây.

Thời gian theo bước tác tử

Bảng theo vai trò nằm ở mục 6.4. Dữ liệu gốc là trường `durationMs` của các bước trong lượt chạy, đọc qua `GET /api/workflows/runs/:runId`; các lượt chạy của lần kiểm thử triển khai được lưu trong `docs/phase7-e2e/out/`.

Phụ lục E. Hướng dẫn sử dụng

Cài đặt bằng Docker

```
        cp .env.production.example .env.production
        # Điền POSTGRES_PASSWORD, JWT_SECRET (openssl rand -hex 32, tối thiểu
32 ký tự),
        # CORS_ORIGIN (địa chỉ web), NINE_ROUTER_URL và NINE_ROUTER_API_KEY
(cổng LLM),
        # SEED_ADMIN_EMAIL và SEED_ADMIN_PASSWORD (tối thiểu 12 ký tự)
        docker compose -f docker-compose.prod.yml --env-file .env.production up
-d --build
```

Giao diện web ở `http://localhost:8080` (đổi bằng `WEB_PORT`). Lần khởi động đầu chạy migration và tạo tài khoản quản trị; sau đó đặt `SEED_ON_START=false`. API từ chối khởi động nếu còn giá trị mẫu (`CHANGE_ME`), thiếu `JWT_SECRET` hoặc `CORS_ORIGIN`, hoặc khoá JWT ngắn hơn 32 ký tự, và in đủ danh sách lỗi. Thêm `--profile demo` để chạy kèm LLM kịch bản phục vụ trình diễn.

Vai trò người dùng

Vai trò	Có thể
Quản trị viên (`admin`)	Mọi thao tác, gồm quản lý người dùng và vai trò, khoá API, webhook, cấu hình mô hình LLM
Thành viên hội đồng (`board_member`)	Tạo và quản lý dự án của mình, duyệt hoặc từ chối phê duyệt, xem báo cáo và phân tích
Người xem (`viewer`)	Chỉ xem; giao diện ẩn các nút tạo, sửa, duyệt và API trả 403 nếu gọi trực tiếp

Các thao tác chính

Giao diện dùng nhãn tiếng Anh; tên màn hình được ghi nguyên văn trong ngoặc.

1. **Tạo dự án:** *Dashboard → New Project*, nhập tên, mô tả yêu cầu, ngân sách, chọn mẫu quy trình (ví dụ dự án phần mềm OOAD hoặc chỉ phân tích và thiết kế) và tham số của mẫu. Quy trình bắt đầu ngay.
2. **Theo dõi:** *Virtual Office* cho thấy trạng thái tám tác tử theo thời gian thực, và *Meeting Room* hiển thị trao đổi giữa các tác tử. Trang chi tiết dự án có tab *Workflow Runs* với dòng thời gian, ảnh chụp và nút khôi phục.

3. **Phê duyệt:** khi quy trình đến nút phê duyệt, thông báo xuất hiện cho thành viên hội đồng; duyệt hoặc từ chối kèm ghi chú trên trang chi tiết dự án.
4. **Xem sản phẩm:** tab *Artifacts* có tóm tắt OOAD, kết quả cổng chất lượng, cây tệp, trình xem Markdown, mã và sơ đồ, và nút tải zip.
5. **Báo cáo và dự báo:** tab *Reports* có chi phí theo vai trò, dự báo thời gian và chi phí, điểm chất lượng mã; xuất JSON, CSV hoặc PDF.
6. **Phân tích tác tử:** *Agent Analytics* so sánh hiệu năng các vai trò và cảnh báo bất thường; *Agent Models* chọn mô hình LLM cho từng vai trò.
7. **Quản trị:** *Users & Roles* (vai trò, kích hoạt), *Audit Log* (lọc, xuất CSV, bất thường), *Integrations* (khoá API, webhook, lịch sử gửi).